

**FIAP — FACULDADE DE INFORMATICA E ADMINISTRACAO PAULISTA**

GUILHERME MACEDO MARTINS  
HENRIQUE MARTINS OLIVEIRA  
PEDRO HENRIQUE LUIZ ALVES DUARTE

**KURAVET: PLATAFORMA DE SAUDE PREVENTIVA CONTINUA  
PARA PETS**

*Modelagem e implementacao de banco de dados Oracle com PL/SQL*

Sao Paulo

2026



## SUMARIO

|                                                                   |    |
|-------------------------------------------------------------------|----|
| 1 INTRODUCAO E OBJETIVO DO PROJETO .....                          | 4  |
| 2 MODELO DESCRITIVO DA ESTRUTURA DE ARMAZENAMENTO.....            | 6  |
| 3 MODELAGEM — DER (NOTACAO DE BARKER) E MER (3FN).....            | 10 |
| 4 SCRIPTS DDL .....                                               | 13 |
| 5 CARGA DE DADOS (PROCEDURES PARAMETRIZADAS E LOG DE ERROS) ..... | 16 |
| 6 CONSULTAS COM JUNCOES, AGRUPAMENTO E ORDENACAO.....             | 23 |
| 7 BLOCO: VALOR ATUAL, ANTERIOR E PROXIMO .....                    | 25 |
| 8 RELATORIOS COM CURSOR EXPLICITO E TOMADA DE DECISAO.....        | 27 |
| 9 FUNCOES E PROCEDIMENTOS DE APOIO .....                          | 31 |
| 10 TRIGGER DE AUDITORIA .....                                     | 39 |
| 11 TRATAMENTO DE EXCECOES — VISAO CONSOLIDADA.....                | 42 |
| 12 CONCLUSAO.....                                                 | 43 |

## 1 INTRODUCAO E OBJETIVO DO PROJETO

O KuraVet e uma plataforma de saude preventiva continua para pets, cujo objetivo e centralizar o historico clinico de animais, conectar tutores, veterinarios e clinicas, e permitir o acompanhamento de consultas e pagamentos ao longo do tempo. Este documento apresenta a modelagem relacional implementada em Oracle Database e toda a logica PL/SQL desenvolvida para o Projeto de Banco de Dados Relacional da disciplina Mastering Relational and Non-Relational Database.

Esta versao da documentacao corresponde a entrega da Sprint 3 e incorpora as correcoes solicitadas no feedback do professor sobre a Sprint 2, adicionando os itens que ainda nao haviam sido desenvolvidos: modelo descritivo da estrutura de armazenamento, modelagem DER/MER, carga de dados por procedures parametrizadas com log de erros, consultas com juncoes agrupadas e ordenadas, o bloco de leitura sequencial (valor atual/anterior/proximo) e os quatro relatorios com cursor explicito e tomada de decisao.

O banco de dados atua como a fonte da verdade (single source of truth) do sistema, sendo consumido por APIs desenvolvidas em Java (Spring) e .NET. Por isso, a modelagem prioriza integridade referencial, constraints de dominio e procedures que expoem dados de forma estruturada e auditavel.

### 1.1 Escopo tecnico atendido nesta entrega

- Modelo descritivo completo da estrutura de armazenamento (6 tabelas).
- Diagrama entidade-relacionamento (DER) em notacao de Barker e Modelo Fisico (MER) normalizado ate a 3a Forma Normal (3FN).
- Scripts DDL executados no Oracle Database, com PK, FK e CHECK CONSTRAINTS.
- 5 procedures de carga de dados (uma por tabela), por passagem de parametro, sem hard-code, com log de erros em tabela dedicada.
- 2 blocos anonimos com, no total, 3 consultas de juncao (JOIN) usando GROUP BY e ORDER BY.
- 1 bloco que exhibe valor atual, anterior e proximo da mesma coluna, com tratamento de bordas ('Vazio').

- 4 blocos anônimos com cursor explícito e tomada de decisão, incluindo um relatório completo (listagem + sumarização + agrupamento).
- Função de conversão para JSON 100% manual, Procedimento de subtotais manuais, Função de cálculo de idade do pet e Trigger de auditoria — herança da Sprint 2, mantidos e validados nesta entrega.
- Tratamento de exceções com no mínimo 3 cláusulas EXCEPTION WHEN distintas (incluindo OTHERS) em todos os blocos nomeados e anônimos.

## 2 MODELO DESCRITIVO DA ESTRUTURA DE ARMAZENAMENTO

Esta seção descreve, tabela a tabela, a estrutura de armazenamento do banco de dados KuraVet: sua finalidade de negócio, colunas, tipos de dados e regras de integridade (chaves primárias, chaves estrangeiras e restrições de domínio). O esquema segue um modelo relacional normalizado, com uma tabela de dimensão para tutores, uma para veterinários, uma tabela de pets vinculada ao tutor, uma tabela de fato transacional de consultas (ligando pet e veterinário), uma tabela de fato de pagamentos vinculada a cada consulta, e uma tabela de auditoria alimentada por trigger.

### 2.1 Visão geral dos relacionamentos

*TUTOR (1) — (N) PET (1) — (N) CONSULTA (N) — (1) VETERINARIO*

*CONSULTA (1) — (N) FATO\_PAGAMENTO      CONSULTA (1) — (N)*  
*AUDITORIA\_LOG (via trigger)*

### 2.2 Tabela TUTOR

Armazena os dados cadastrais dos tutores responsáveis pelos pets. CPF é único (UNIQUE) para evitar cadastros duplicados.

Quadro 1 — Estrutura da tabela TUTOR

| Coluna        | Tipo          | Regra                     |
|---------------|---------------|---------------------------|
| ID_TUTOR      | NUMBER(6)     | PK                        |
| NOME          | VARCHAR2(100) | NOT NULL                  |
| CPF           | VARCHAR2(14)  | NOT NULL, UNIQUE          |
| TELEFONE      | VARCHAR2(20)  | -                         |
| EMAIL         | VARCHAR2(100) | -                         |
| ENDERECO      | VARCHAR2(200) | -                         |
| DATA_CADASTRO | DATE          | DEFAULT SYSDATE, NOT NULL |

Fonte: elaborado pelos autores a partir do script\_bd.sql (2026).

### 2.3 Tabela VETERINARIO

Cadastro dos profissionais veterinários, com CRMV único e especialidade para segmentar o atendimento.

Quadro 2 — Estrutura da tabela VETERINARIO

| Coluna         | Tipo          | Regra            |
|----------------|---------------|------------------|
| ID_VETERINARIO | NUMBER(6)     | PK               |
| NOME           | VARCHAR2(100) | NOT NULL         |
| CRMV           | VARCHAR2(20)  | NOT NULL, UNIQUE |
| ESPECIALIDADE  | VARCHAR2(60)  | -                |
| TELEFONE       | VARCHAR2(20)  | -                |
| EMAIL          | VARCHAR2(100) | -                |

Fonte: elaborado pelos autores a partir do script\_bd.sql (2026).

## 2.4 Tabela PET

Cada pet pertence a um unico tutor (FK ID\_TUTOR). A coluna SEXO possui CHECK CONSTRAINT restringindo os valores a 'M' ou 'F'.

Quadro 3 — Estrutura da tabela PET

| Coluna          | Tipo         | Regra              |
|-----------------|--------------|--------------------|
| ID_PET          | NUMBER(6)    | PK                 |
| NOME            | VARCHAR2(60) | NOT NULL           |
| ESPECIE         | VARCHAR2(30) | NOT NULL           |
| RACA            | VARCHAR2(60) | -                  |
| DATA_NASCIMENTO | DATE         | NOT NULL           |
| SEXO            | CHAR(1)      | CHECK IN ('M','F') |
| ID_TUTOR        | NUMBER(6)    | FK -> TUTOR        |

Fonte: elaborado pelos autores a partir do script\_bd.sql (2026).

## 2.5 Tabela CONSULTA

Tabela de fato que registra cada atendimento, ligando um PET a um VETERINARIO. O STATUS e restrito por CHECK CONSTRAINT a tres valores possiveis, evitando dados inconsistentes vindos das APIs consumidoras.

Quadro 4 — Estrutura da tabela CONSULTA

| Coluna      | Tipo      | Regra |
|-------------|-----------|-------|
| ID_CONSULTA | NUMBER(6) | PK    |

|                |               |                                               |
|----------------|---------------|-----------------------------------------------|
| ID_PET         | NUMBER(6)     | FK -> PET                                     |
| ID_VETERINARIO | NUMBER(6)     | FK -> VETERINARIO                             |
| DATA_CONSULTA  | DATE          | NOT NULL                                      |
| TIPO_CONSULTA  | VARCHAR2(40)  | NOT NULL                                      |
| DIAGNOSTICO    | VARCHAR2(400) | -                                             |
| STATUS         | VARCHAR2(20)  | CHECK IN ('AGENDADA','REALIZADA','CANCELADA') |

Fonte: elaborado pelos autores a partir do script\_bd.sql (2026).

## 2.6 Tabela FATO\_PAGAMENTO

Tabela de fato financeira usada no calculo de subtotais (Secao 9.3) e nos relatorios de sumarizacao (Secao 8). Contem duas colunas categoricas (CLINICA e TIPO\_PAGAMENTO) e uma coluna numerica (VALOR).

Quadro 5 — Estrutura da tabela FATO\_PAGAMENTO

| Coluna         | Tipo         | Regra                   |
|----------------|--------------|-------------------------|
| ID_PAGAMENTO   | NUMBER(6)    | PK                      |
| ID_CONSULTA    | NUMBER(6)    | FK -> CONSULTA          |
| CLINICA        | VARCHAR2(60) | NOT NULL (categorica 1) |
| TIPO_PAGAMENTO | VARCHAR2(30) | NOT NULL (categorica 2) |
| VALOR          | NUMBER(10,2) | NOT NULL, CHECK >= 0    |
| DATA_PAGAMENTO | DATE         | DEFAULT SYSDATE         |

Fonte: elaborado pelos autores a partir do script\_bd.sql (2026).

## 2.7 Tabela AUDITORIA\_LOG

Tabela dedicada a registrar automaticamente as operacoes DML realizadas em CONSULTA, alimentada pela trigger TRG\_AUDITORIA\_CONSULTA (Secao 10).

Quadro 6 — Estrutura da tabela AUDITORIA\_LOG

| Coluna   | Tipo         | Regra                                  |
|----------|--------------|----------------------------------------|
| ID_LOG   | NUMBER       | PK, GENERATED ALWAYS AS IDENTITY       |
| USUARIO  | VARCHAR2(60) | usuario Oracle que executou a operacao |
| OPERACAO | VARCHAR2(10) | INSERT / UPDATE / DELETE               |

|                |                |                             |
|----------------|----------------|-----------------------------|
| DATA_HORA      | DATE           | momento da operacao         |
| TABELA_AFETADA | VARCHAR2(30)   | nome da tabela monitorada   |
| VALORES_OLD    | VARCHAR2(4000) | valores antes da alteracao  |
| VALORES_NEW    | VARCHAR2(4000) | valores depois da alteracao |

Fonte: elaborado pelos autores a partir do script\_bd.sql (2026).

## 2.8 Tabela LOG\_ERRO\_CARGA (nova nesta Sprint)

Criada para atender ao requisito de carga de dados: registra qualquer falha ocorrida durante a execucao das procedures de carga (Secao 5), armazenando o nome da procedure, o usuario, a data/hora da ocorrencia, o codigo do erro e a mensagem de erro.

Quadro 7 — Estrutura da tabela LOG\_ERRO\_CARGA

| Coluna           | Tipo          | Regra                                      |
|------------------|---------------|--------------------------------------------|
| ID_LOG_ERRO      | NUMBER        | PK, GENERATED ALWAYS AS IDENTITY           |
| NM_PROCEDURE     | VARCHAR2(60)  | NOT NULL — nome da procedure que falhou    |
| NM_USUARIO       | VARCHAR2(60)  | NOT NULL — usuario Oracle (USER)           |
| DT_OCORRENCIA    | DATE          | DEFAULT SYSDATE, NOT NULL                  |
| CD_ERRO          | VARCHAR2(20)  | NOT NULL — SQLCODE ou codigo customizado   |
| DS_MENSAGEM_ERRO | VARCHAR2(500) | NOT NULL — SQLERRM ou mensagem customizada |

Fonte: elaborado pelos autores a partir do script\_bd.sql (2026).

### 3 MODELAGEM — DER (NOTACAO DE BARKER) E MER (3FN)

Os diagramas desta secao foram construidos no Oracle SQL Developer Data Modeler, conforme exigido no enunciado, representando o Modelo Logico (DER, notacao de Barker) e o Modelo Fisico (MER) do KuraVet.

#### 3.1 Modelo Logico — DER (notacao de Barker)

No DER a seguir, cada entidade e um retangulo com o nome no cabecalho; os atributos de chave primaria sao marcados com '#' e as chaves estrangeiras com 'FK'; os relacionamentos entre CONSULTA e as demais entidades (PET, VETERINARIO, FATO\_PAGAMENTO) sao representados com a cardinalidade de Barker (tracos para o lado 'um' obrigatorio e pe-de-galinha para o lado 'muitos').

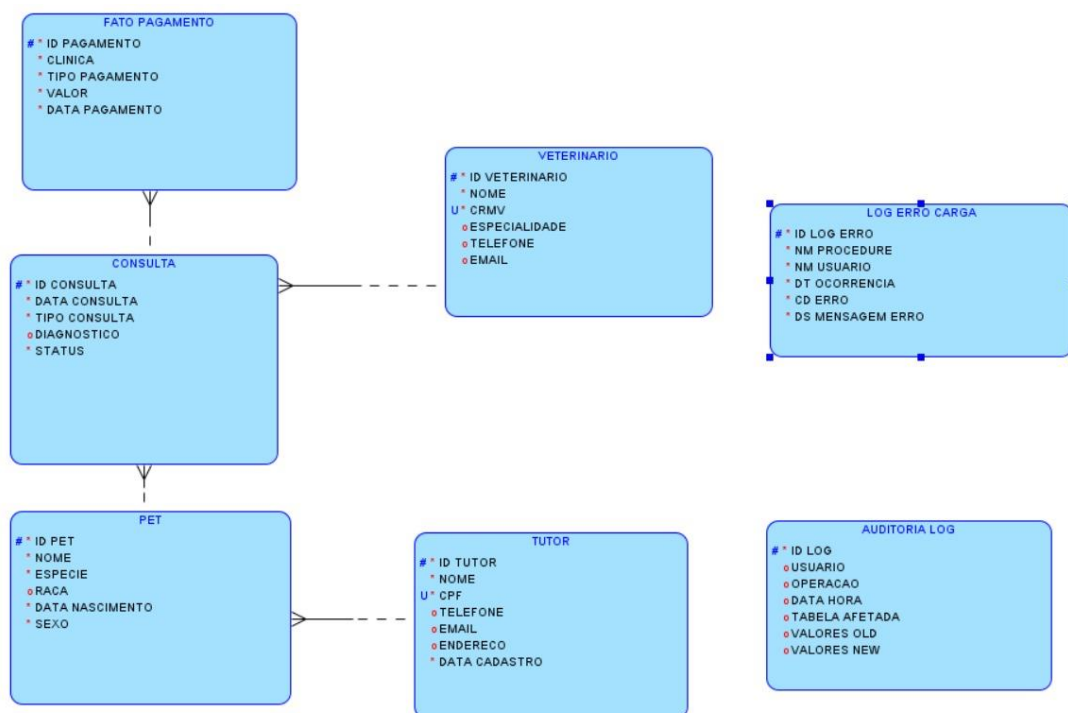


Figura 1 — Diagrama Entidade-Relacionamento (DER) do KuraVet, notacao de Barker — Oracle Data Modeler

Fonte: gerado pelos autores no Oracle SQL Developer Data Modeler (2026).

#### 3.2 Modelo Fisico — MER

O Modelo de Entidade-Relacionamento Fisico (MER) a seguir corresponde exatamente as tabelas criadas no Oracle Database por meio dos scripts DDL apresentados na Secao 4: mesmos nomes de tabelas, colunas, tipos de dados ('NUMBER(6)', 'VARCHAR2(n)', 'DATE'), chaves primarias (P) e chaves estrangeiras (F), incluindo as sete tabelas do projeto

— TUTOR, VETERINARIO, PET, CONSULTA, FATO\_PAGAMENTO, AUDITORIA\_LOG e LOG\_ERRO\_CARGA.

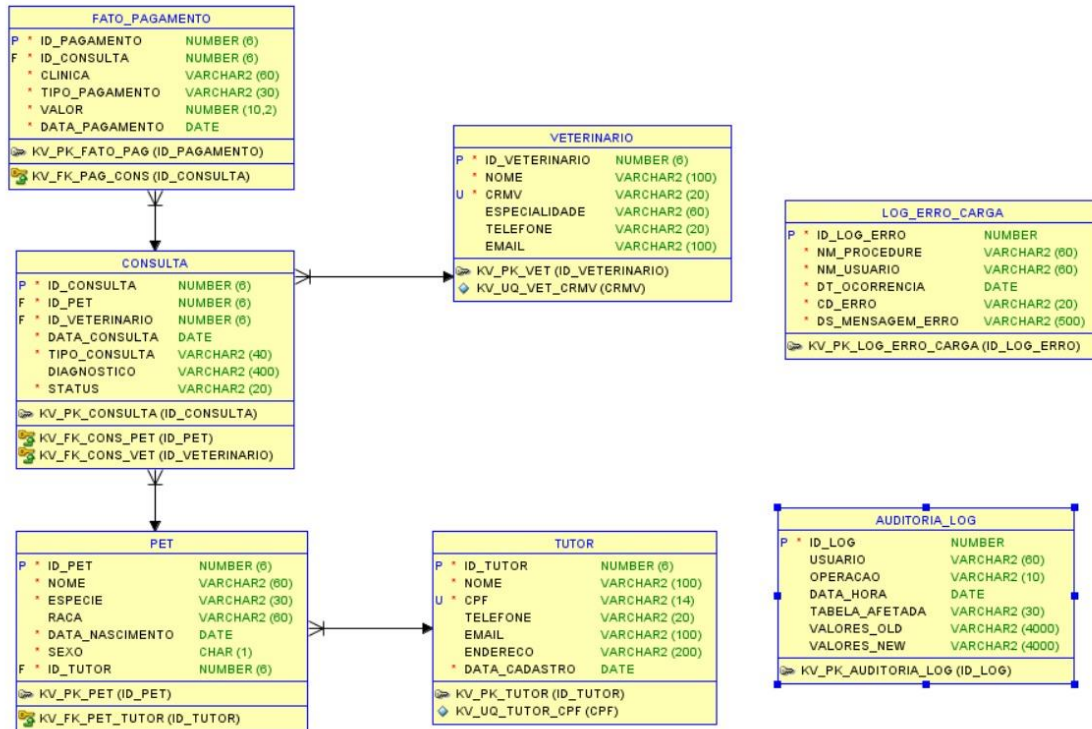


Figura 2 — Modelo de Entidade-Relacionamento Físico (MER) do KuraVet — Oracle Data Modeler

Fonte: gerado pelos autores no Oracle SQL Developer Data Modeler (2026).

Não há divergência entre o modelo físico e o banco de dados em produção, pois o DDL apresentado na Seção 4 foi executado diretamente a partir dessa mesma estrutura.

### 3.3 Justificativa da 3ª Forma Normal (3FN)

O esquema atende a 3FN porque, em cada tabela:

- 1FN — todos os atributos são atômicos (não há grupos repetitivos nem colunas multivaloradas); cada tabela possui uma chave primária que identifica unicamente cada linha (ID\_TUTOR, ID\_VETERINARIO, ID\_PET, ID\_CONSULTA, ID\_PAGAMENTO, ID\_LOG, ID\_LOG\_ERRO).
- 2FN — como todas as chaves primárias são simples (não compostas), não há dependência parcial possível; todo atributo não-chave depende da chave primária inteira.
- 3FN — não há dependências transitivas: por exemplo, em CONSULTA, o TIPO\_CONSULTA e o STATUS dependem apenas de ID\_CONSULTA, e não de

ID\_PET ou ID\_VETERINARIO; em FATO\_PAGAMENTO, o VALOR e a CLINICA dependem apenas de ID\_PAGAMENTO. Dados que poderiam gerar dependencia transitiva (como nome do tutor em CONSULTA) foram propositalmente mantidos apenas nas tabelas TUTOR e PET, sendo obtidos via JOIN quando necessario (Secao 6).

Dessa forma, o modelo elimina redundancias e anomalias de insercao, atualizacao e exclusao, cumprindo o requisito minimo de normalizacao ate a 3FN.

## 4 SCRIPTS DDL

Os comandos DDL abaixo correspondem exatamente ao Modelo Físico descrito na Seção 3 e ao Modelo Descritivo da Seção 2. Todas as tabelas foram efetivamente criadas e executadas com sucesso no Oracle Database, com PRIMARY KEY, FOREIGN KEY e CHECK CONSTRAINTS nomeadas com o prefixo KV\_ (KuraVet), evitando colisão de nomes de constraint no schema.

### 4.1 Comandos CREATE TABLE

-- Tabela TUTOR

```
CREATE TABLE TUTOR (
    ID_TUTOR      NUMBER(6)      NOT NULL,
    NOME          VARCHAR2(100)  NOT NULL,
    CPF           VARCHAR2(14)   NOT NULL,
    TELEFONE      VARCHAR2(20),
    EMAIL         VARCHAR2(100),
    ENDERECO      VARCHAR2(200),
    DATA_CADASTRO DATE          DEFAULT SYSDATE NOT NULL,
    CONSTRAINT KV_PK_TUTOR PRIMARY KEY (ID_TUTOR),
    CONSTRAINT KV_UQ_TUTOR_CPF UNIQUE (CPF)
);
```

-- Tabela VETERINARIO

```
CREATE TABLE VETERINARIO (
    ID_VETERINARIO NUMBER(6)      NOT NULL,
    NOME          VARCHAR2(100)  NOT NULL,
    CRMV          VARCHAR2(20)   NOT NULL,
    ESPECIALIDADE VARCHAR2(60),
    TELEFONE      VARCHAR2(20),
```

```

EMAIL          VARCHAR2(100),

CONSTRAINT KV_PK_VET PRIMARY KEY (ID_VETERINARIO),

CONSTRAINT KV_UQ_VET_CRMV UNIQUE (CRMV)

);

-- Tabela PET

CREATE TABLE PET (

    ID_PET          NUMBER(6)          NOT NULL,

    NOME            VARCHAR2(60)       NOT NULL,

    ESPECIE         VARCHAR2(30)       NOT NULL,

    RACA            VARCHAR2(60),

    DATA_NASCIMENTO DATE              NOT NULL,

    SEXO            CHAR(1)            NOT NULL,

    ID_TUTOR        NUMBER(6)          NOT NULL,

    CONSTRAINT KV_PK_PET PRIMARY KEY (ID_PET),

    CONSTRAINT KV_CK_PET_SEXO CHECK (SEXO IN ('M','F')),

    CONSTRAINT KV_FK_PET_TUTOR FOREIGN KEY (ID_TUTOR) REFERENCES TUTOR(ID_TUTOR)

);

-- Tabela CONSULTA

CREATE TABLE CONSULTA (

    ID_CONSULTA     NUMBER(6)          NOT NULL,

    ID_PET          NUMBER(6)          NOT NULL,

    ID_VETERINARIO  NUMBER(6)          NOT NULL,

    DATA_CONSULTA  DATE              NOT NULL,

    TIPO_CONSULTA   VARCHAR2(40)       NOT NULL,

    DIAGNOSTICO     VARCHAR2(400),

```

```

STATUS          VARCHAR2(20)    DEFAULT 'REALIZADA' NOT NULL,

CONSTRAINT KV_PK_CONSULTA PRIMARY KEY (ID_CONSULTA),

CONSTRAINT KV_CK_CONS_STATUS CHECK (STATUS IN ('AGENDADA','REALIZADA','CANCELADA')),

CONSTRAINT KV_FK_CONS_PET FOREIGN KEY (ID_PET) REFERENCES PET(ID_PET),

CONSTRAINT KV_FK_CONS_VET FOREIGN KEY (ID_VETERINARIO) REFERENCES
VETERINARIO(ID_VETERINARIO)

);

```

```
-- Tabela FATO_PAGAMENTO
```

```

CREATE TABLE FATO_PAGAMENTO (

    ID_PAGAMENTO    NUMBER(6)      NOT NULL,

    ID_CONSULTA     NUMBER(6)      NOT NULL,

    CLINICA         VARCHAR2(60)   NOT NULL,

    TIPO_PAGAMENTO  VARCHAR2(30)   NOT NULL,

    VALOR           NUMBER(10,2)   NOT NULL,

    DATA_PAGAMENTO DATE           DEFAULT SYSDATE NOT NULL,

    CONSTRAINT KV_PK_FATO_PAG PRIMARY KEY (ID_PAGAMENTO),

    CONSTRAINT KV_CK_PAG_VALOR CHECK (VALOR >= 0),

    CONSTRAINT KV_FK_PAG_CONS FOREIGN KEY (ID_CONSULTA) REFERENCES CONSULTA(ID_CONSULTA)

);

```

```
-- Tabela AUDITORIA_LOG
```

```

CREATE TABLE AUDITORIA_LOG (

    ID_LOG          NUMBER GENERATED ALWAYS AS IDENTITY,

    USUARIO         VARCHAR2(60),

    OPERACAO        VARCHAR2(10),

    DATA_HORA      DATE,

```

```

TABELA_AFETADA  VARCHAR2(30),

VALORES_OLD     VARCHAR2(4000),

VALORES_NEW     VARCHAR2(4000),

CONSTRAINT KV_PK_AUDITORIA_LOG PRIMARY KEY (ID_LOG)

);

-- Tabela LOG_ERRO_CARGA (Sprint 3)

CREATE TABLE LOG_ERRO_CARGA (

    ID_LOG_ERRO      NUMBER GENERATED ALWAYS AS IDENTITY,

    NM_PROCEDURE     VARCHAR2(60)    NOT NULL,

    NM_USUARIO       VARCHAR2(60)    NOT NULL,

    DT_OCORRENCIA    DATE            DEFAULT SYSDATE NOT NULL,

    CD_ERRO          VARCHAR2(20)    NOT NULL,

    DS_MENSAGEM_ERRO VARCHAR2(500)    NOT NULL,

    CONSTRAINT KV_PK_LOG_ERRO_CARGA PRIMARY KEY (ID_LOG_ERRO)

);

```

## 4.2 Sequences de apoio as procedures de carga

Para que as procedures de carga de dados (Secao 5) gerem chaves primarias sem hard-code e sem colidir com os IDs 1 a 10 ja inseridos na carga inicial, foram criadas sequences dedicadas, uma por tabela, iniciando em 100:

```

CREATE SEQUENCE SEQ_KV_TUTOR          START WITH 100 INCREMENT BY 1 NOCACHE;

CREATE SEQUENCE SEQ_KV_VETERINARIO    START WITH 100 INCREMENT BY 1 NOCACHE;

CREATE SEQUENCE SEQ_KV_PET            START WITH 100 INCREMENT BY 1 NOCACHE;

CREATE SEQUENCE SEQ_KV_CONSULTA       START WITH 100 INCREMENT BY 1 NOCACHE;

CREATE SEQUENCE SEQ_KV_PAGAMENTO      START WITH 100 INCREMENT BY 1 NOCACHE;

```

O arquivo completo (2TDSPO\_2024\_CodigoSql\_Integrantes.sql) contem tambem os blocos de limpeza idempotente (DROP ... EXCEPTION WHEN OTHERS THEN NULL) que

antecedem cada CREATE, permitindo executar o script do zero quantas vezes forem necessarias sem erro de objeto ja existente.

## 5 CARGA DE DADOS (PROCEDURES PARAMETRIZADAS E LOG DE ERROS)

A carga inicial do banco (10 registros por tabela, detalhada na Secao 2) foi realizada por comandos INSERT diretos, documentados e evidenciados nas Secoes 9 e 10. Para atender ao requisito especifico desta Sprint, o grupo desenvolveu adicionalmente uma PROCEDURE de carga por tabela, recebendo os dados exclusivamente por passagem de parametro (sem hard-code no corpo da procedure), com tratamento de excecao e registro de falhas na tabela LOG\_ERRO\_CARGA (Secao 2.8).

### 5.1 Padrao adotado em todas as procedures de carga

- Parametros IN tipados com %TYPE, referenciando diretamente a coluna da tabela de destino (sem hard-code de tipos ou valores).
- Geracao da chave primaria via SEQUENCE dedicada (Secao 4.2), nunca fixada no codigo.
- Ao menos uma regra de negocio validada antes do INSERT (ex.: existencia da FK, formato do dado, dominio do valor).
- No minimo 3 clausulas EXCEPTION WHEN distintas por procedure, sempre incluindo WHEN OTHERS.
- Toda excecao grava um registro em LOG\_ERRO\_CARGA com nome da procedure, USER, SYSDATE, codigo do erro e mensagem.

### 5.2 PRC\_CARGA\_TUTOR

```
CREATE OR REPLACE PROCEDURE PRC_CARGA_TUTOR (

    p_nome      IN TUTOR.NOME%TYPE,

    p_cpf       IN TUTOR.CPF%TYPE,

    p_telefone  IN TUTOR.TELEFONE%TYPE,

    p_email     IN TUTOR.EMAIL%TYPE,

    p_endereco  IN TUTOR.ENDERECO%TYPE

) IS

    v_id_tutor   TUTOR.ID_TUTOR%TYPE;
```

```

        e_cpf_invalido EXCEPTION;

BEGIN

    -- Regra de negocio: CPF deve ser informado e ter ao menos 11 digitos numericos

    IF p_cpf IS NULL OR LENGTH(REGEXP_REPLACE(p_cpf, '^[0-9]', '')) < 11 THEN

        RAISE e_cpf_invalido;

    END IF;

    v_id_tutor := SEQ_KV_TUTOR.NEXTVAL;

    INSERT INTO TUTOR (ID_TUTOR, NOME, CPF, TELEFONE, EMAIL, ENDERECO, DATA_CADASTRO)

    VALUES (v_id_tutor, p_nome, p_cpf, p_telefone, p_email, p_endereco, SYSDATE);

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('OK: Tutor "' || p_nome || '" cadastrado com ID_TUTOR = ' ||
v_id_tutor);

EXCEPTION

    WHEN e_cpf_invalido THEN

        INSERT INTO LOG_ERRO_CARGA (NM_PROCEDURE, NM_USUARIO, DT_OCORRENCIA, CD_ERRO,
DS_MENSAGEM_ERRO)

        VALUES ('PRC_CARGA_TUTOR', USER, SYSDATE, '-20001',

                'CPF invalido ou nao informado para o tutor ' || NVL(p_nome, '(nome nao
informado)'));

        DBMS_OUTPUT.PUT_LINE('FALHA: CPF invalido para o tutor "' || p_nome || '"');

    WHEN DUP_VAL_ON_INDEX THEN

        INSERT INTO LOG_ERRO_CARGA (NM_PROCEDURE, NM_USUARIO, DT_OCORRENCIA, CD_ERRO,
DS_MENSAGEM_ERRO)

```

```

VALUES ('PRC_CARGA_TUTOR', USER, SYSDATE, SQLCODE, SUBSTR('CPF ja cadastrado: ' ||
SQLERRM, 1, 480));

DBMS_OUTPUT.PUT_LINE('FALHA: CPF ja cadastrado para o tutor "' || p_nome || '".');

WHEN OTHERS THEN

INSERT INTO LOG_ERRO_CARGA (NM_PROCEDURE, NM_USUARIO, DT_OCORRENCIA, CD_ERRO,
DS_MENSAGEM_ERRO)

VALUES ('PRC_CARGA_TUTOR', USER, SYSDATE, SQLCODE, SUBSTR(SQLERRM, 1, 480));

DBMS_OUTPUT.PUT_LINE('FALHA inesperada ao cadastrar tutor "' || p_nome || '" : ' ||
SQLERRM);

END PRC_CARGA_TUTOR;

/

```

Regra de negocio validada: CPF obrigatorio com no minimo 11 digitos numericos. Excecoes tratadas: e\_cpf\_invalido (customizada), DUP\_VAL\_ON\_INDEX (CPF duplicado, protegido pela UNIQUE da Secao 2.2) e OTHERS.

### 5.3 PRC\_CARGA\_VETERINARIO

```

CREATE OR REPLACE PROCEDURE PRC_CARGA_VETERINARIO (

    p_nome          IN VETERINARIO.NOME%TYPE,

    p_crmv          IN VETERINARIO.CRMV%TYPE,

    p_especialidade IN VETERINARIO.ESPECIALIDADE%TYPE,

    p_telefone      IN VETERINARIO.TELEFONE%TYPE,

    p_email         IN VETERINARIO.EMAIL%TYPE

) IS

    v_id_vet        VETERINARIO.ID_VETERINARIO%TYPE;

    e_crmv_invalido EXCEPTION;

BEGIN

    -- Regra de negocio: CRMV obrigatorio, com no minimo 5 caracteres

    IF p_crmv IS NULL OR LENGTH(TRIM(p_crmv)) < 5 THEN

        RAISE e_crmv_invalido;
    
```

```

END IF;

v_id_vet := SEQ_KV_VETERINARIO.NEXTVAL;

INSERT INTO VETERINARIO (ID_VETERINARIO, NOME, CRMV, ESPECIALIDADE, TELEFONE, EMAIL)
VALUES (v_id_vet, p_nome, p_crmv, p_especialidade, p_telefone, p_email);

COMMIT;

DBMS_OUTPUT.PUT_LINE('OK: Veterinario "' || p_nome || '" cadastrado com ID_VETERINARIO
= ' || v_id_vet);

EXCEPTION

WHEN e_crmv_invalido THEN

    INSERT INTO LOG_ERRO_CARGA (NM_PROCEDURE, NM_USUARIO, DT_OCORRENCIA, CD_ERRO,
DS_MENSAGEM_ERRO)

    VALUES ('PRC_CARGA_VETERINARIO', USER, SYSDATE, '-20002',

            'CRMV invalido ou nao informado para o veterinario ' || NVL(p_nome, '(nome
nao informado)'));

    DBMS_OUTPUT.PUT_LINE('FALHA: CRMV invalido para o veterinario "' || p_nome ||
'".');

WHEN DUP_VAL_ON_INDEX THEN

    INSERT INTO LOG_ERRO_CARGA (NM_PROCEDURE, NM_USUARIO, DT_OCORRENCIA, CD_ERRO,
DS_MENSAGEM_ERRO)

    VALUES ('PRC_CARGA_VETERINARIO', USER, SYSDATE, SQLCODE, SUBSTR('CRMV ja
cadastrado: ' || SQLERRM, 1, 480));

    DBMS_OUTPUT.PUT_LINE('FALHA: CRMV ja cadastrado para o veterinario "' || p_nome ||
'".');

WHEN OTHERS THEN

    INSERT INTO LOG_ERRO_CARGA (NM_PROCEDURE, NM_USUARIO, DT_OCORRENCIA, CD_ERRO,
DS_MENSAGEM_ERRO)

```

```

VALUES ('PRC_CARGA_VETERINARIO', USER, SYSDATE, SQLCODE, SUBSTR(SQLERRM, 1, 480));

DBMS_OUTPUT.PUT_LINE('FALHA inesperada ao cadastrar veterinario "' || p_nome || '" :
' || SQLERRM);

END PRC_CARGA_VETERINARIO;

/

```

Regra de negocio validada: CRMV obrigatorio com no minimo 5 caracteres. Excecoes tratadas: e\_crmv\_invalido (customizada), DUP\_VAL\_ON\_INDEX (CRMV duplicado, protegido pela UNIQUE da Secao 2.3) e OTHERS.

## 5.4 PRC\_CARGA\_PET

```

CREATE OR REPLACE PROCEDURE PRC_CARGA_PET (

    p_nome          IN PET.NOME%TYPE,

    p_especie       IN PET.ESPECIE%TYPE,

    p_raca          IN PET.RACA%TYPE,

    p_data_nascimento IN PET.DATA_NASCIMENTO%TYPE,

    p_sexo          IN PET.SEXO%TYPE,

    p_id_tutor      IN PET.ID_TUTOR%TYPE

) IS

    v_id_pet        PET.ID_PET%TYPE;

    v_qtd_tutor     NUMBER;

    e_tutor_inexistente EXCEPTION;

    e_data_nasc_futura EXCEPTION;

BEGIN

    -- Regra de negocio 1: o tutor referenciado precisa existir

    SELECT COUNT(*) INTO v_qtd_tutor FROM TUTOR WHERE id_tutor = p_id_tutor;

    IF v_qtd_tutor = 0 THEN

        RAISE e_tutor_inexistente;

    END IF;

```

```

-- Regra de negocio 2: data de nascimento nao pode ser no futuro

IF p_data_nascimento > SYSDATE THEN

    RAISE e_data_nasc_futura;

END IF;

v_id_pet := SEQ_KV_PET.NEXTVAL;

INSERT INTO PET (ID_PET, NOME, ESPECIE, RACA, DATA_NASCIMENTO, SEXO, ID_TUTOR)

VALUES (v_id_pet, p_nome, p_especie, p_raca, p_data_nascimento, p_sexo, p_id_tutor);

COMMIT;

DBMS_OUTPUT.PUT_LINE('OK: Pet '' || p_nome || '' cadastrado com ID_PET = ' ||
v_id_pet);

EXCEPTION

    WHEN e_tutor_inexistente THEN

        INSERT INTO LOG_ERRO_CARGA (NM_PROCEDURE, NM_USUARIO, DT_OCORRENCIA, CD_ERRO,
DS_MENSAGEM_ERRO)

        VALUES ('PRC_CARGA_PET', USER, SYSDATE, '-20003',

                'ID_TUTOR ' || p_id_tutor || ' nao existe para o pet ' || NVL(p_nome,
'(nome nao informado)'));

        DBMS_OUTPUT.PUT_LINE('FALHA: tutor inexistente para o pet '' || p_nome || ''');

    WHEN e_data_nasc_futura THEN

        INSERT INTO LOG_ERRO_CARGA (NM_PROCEDURE, NM_USUARIO, DT_OCORRENCIA, CD_ERRO,
DS_MENSAGEM_ERRO)

        VALUES ('PRC_CARGA_PET', USER, SYSDATE, '-20004',

```

```

        'Data de nascimento futura informada para o pet ' || NVL(p_nome, '(nome nao
informado)'));

        DBMS_OUTPUT.PUT_LINE('FALHA: data de nascimento futura para o pet "' || p_nome ||
'".');

    WHEN OTHERS THEN

        INSERT INTO LOG_ERRO_CARGA (NM_PROCEDURE, NM_USUARIO, DT_OCORRENCIA, CD_ERRO,
DS_MENSAGEM_ERRO)

        VALUES ('PRC_CARGA_PET', USER, SYSDATE, SQLCODE, SUBSTR(SQLERRM, 1, 480));

        DBMS_OUTPUT.PUT_LINE('FALHA inesperada ao cadastrar pet "' || p_nome || '": ' ||
SQLERRM);

    END PRC_CARGA_PET;

/

```

Regras de negocio validadas: existencia previa do tutor (FK) e data de nascimento nao futura. Excecoes tratadas: e\_tutor\_inexistente, e\_data\_nasc\_futura e OTHERS.

## 5.5 PRC\_CARGA\_CONSULTA

```

CREATE OR REPLACE PROCEDURE PRC_CARGA_CONSULTA (

    p_id_pet          IN CONSULTA.ID_PET%TYPE,

    p_id_veterinario IN CONSULTA.ID_VETERINARIO%TYPE,

    p_data_consulta  IN CONSULTA.DATA_CONSULTA%TYPE,

    p_tipo_consulta  IN CONSULTA.TIPO_CONSULTA%TYPE,

    p_diagnostico    IN CONSULTA.DIAGNOSTICO%TYPE,

    p_status         IN CONSULTA.STATUS%TYPE

) IS

    v_id_consulta     CONSULTA.ID_CONSULTA%TYPE;

    v_qtd_pet         NUMBER;

    e_pet_inexistente EXCEPTION;

    e_status_invalido EXCEPTION;

BEGIN

```

```

-- Regra de negocio 1: o pet referenciado precisa existir

SELECT COUNT(*) INTO v_qtd_pet FROM PET WHERE id_pet = p_id_pet;

IF v_qtd_pet = 0 THEN

    RAISE e_pet_inexistente;

END IF;


-- Regra de negocio 2: status deve pertencer ao dominio valido

IF p_status NOT IN ('AGENDADA', 'REALIZADA', 'CANCELADA') THEN

    RAISE e_status_invalido;

END IF;


v_id_consulta := SEQ_KV_CONSULTA.NEXTVAL;


INSERT INTO CONSULTA (ID_CONSULTA, ID_PET, ID_VETERINARIO, DATA_CONSULTA,
TIPO_CONSULTA, DIAGNOSTICO, STATUS)

VALUES (v_id_consulta, p_id_pet, p_id_veterinario, p_data_consulta, p_tipo_consulta,
p_diagnostico, p_status);


COMMIT;


DBMS_OUTPUT.PUT_LINE('OK: Consulta cadastrada com ID_CONSULTA = ' || v_id_consulta);


EXCEPTION

WHEN e_pet_inexistente THEN

    INSERT INTO LOG_ERRO_CARGA (NM_PROCEDURE, NM_USUARIO, DT_OCORRENCIA, CD_ERRO,
DS_MENSAGEM_ERRO)

VALUES ('PRC_CARGA_CONSULTA', USER, SYSDATE, '-20005',

'ID_PET ' || p_id_pet || ' nao existe para a nova consulta');

```

```

        DBMS_OUTPUT.PUT_LINE('FALHA: pet inexistente (ID_PET = ' || p_id_pet || ').');

    WHEN e_status_invalido THEN

        INSERT INTO LOG_ERRO_CARGA (NM_PROCEDURE, NM_USUARIO, DT_OCORRENCIA, CD_ERRO,
DS_MENSAGEM_ERRO)

        VALUES ('PRC_CARGA_CONSULTA', USER, SYSDATE, '-20006',

                'Status invalido informado: ' || NVL(p_status, '(nulo)'));

        DBMS_OUTPUT.PUT_LINE('FALHA: status invalido "' || p_status || '".');

    WHEN OTHERS THEN

        INSERT INTO LOG_ERRO_CARGA (NM_PROCEDURE, NM_USUARIO, DT_OCORRENCIA, CD_ERRO,
DS_MENSAGEM_ERRO)

        VALUES ('PRC_CARGA_CONSULTA', USER, SYSDATE, SQLCODE, SUBSTR(SQLERRM, 1, 480));

        DBMS_OUTPUT.PUT_LINE('FALHA inesperada ao cadastrar consulta: ' || SQLERRM);

END PRC_CARGA_CONSULTA;

/

```

Regras de negocio validadas: existencia previa do pet (FK) e dominio valido do STATUS. Excecoes tratadas: e\_pet\_inexistente, e\_status\_invalido e OTHERS.

## 5.6 PRC\_CARGA\_PAGAMENTO

```

CREATE OR REPLACE PROCEDURE PRC_CARGA_PAGAMENTO (

    p_id_consulta      IN FATO_PAGAMENTO.ID_CONSULTA%TYPE,

    p_clinica          IN FATO_PAGAMENTO.CLINICA%TYPE,

    p_tipo_pagamento  IN FATO_PAGAMENTO.TIPO_PAGAMENTO%TYPE,

    p_valor            IN FATO_PAGAMENTO.VALOR%TYPE,

    p_data_pagamento  IN FATO_PAGAMENTO.DATA_PAGAMENTO%TYPE

) IS

    v_id_pagamento    FATO_PAGAMENTO.ID_PAGAMENTO%TYPE;

    v_qtd_consulta     NUMBER;

    e_consulta_inexiste EXCEPTION;

    e_valor_invalido   EXCEPTION;

```

```
BEGIN
```

```
-- Regra de negocio 1: a consulta referenciada precisa existir
```

```
SELECT COUNT(*) INTO v_qtd_consulta FROM CONSULTA WHERE id_consulta = p_id_consulta;
```

```
IF v_qtd_consulta = 0 THEN
```

```
    RAISE e_consulta_inexiste;
```

```
END IF;
```

```
-- Regra de negocio 2: valor do pagamento deve ser positivo
```

```
IF p_valor IS NULL OR p_valor <= 0 THEN
```

```
    RAISE e_valor_invalido;
```

```
END IF;
```

```
v_id_pagamento := SEQ_KV_PAGAMENTO.NEXTVAL;
```

```
INSERT INTO FATO_PAGAMENTO (ID_PAGAMENTO, ID_CONSULTA, CLINICA, TIPO_PAGAMENTO, VALOR,  
DATA_PAGAMENTO)
```

```
VALUES (v_id_pagamento, p_id_consulta, p_clinica, p_tipo_pagamento, p_valor,  
p_data_pagamento);
```

```
COMMIT;
```

```
DBMS_OUTPUT.PUT_LINE('OK: Pagamento cadastrado com ID_PAGAMENTO = ' || v_id_pagamento);
```

```
EXCEPTION
```

```
WHEN e_consulta_inexiste THEN
```

```
INSERT INTO LOG_ERRO_CARGA (NM_PROCEDURE, NM_USUARIO, DT_OCORRENCIA, CD_ERRO,  
DS_MENSAGEM_ERRO)
```

```
VALUES ('PRC_CARGA_PAGAMENTO', USER, SYSDATE, '-20007',
```

```

        'ID_CONSULTA ' || p_id_consulta || ' nao existe para o pagamento');

        DBMS_OUTPUT.PUT_LINE('FALHA: consulta inexistente (ID_CONSULTA = ' || p_id_consulta
        || ').');

    WHEN e_valor_invalido THEN

        INSERT INTO LOG_ERRO_CARGA (NM_PROCEDURE, NM_USUARIO, DT_OCORRENCIA, CD_ERRO,
        DS_MENSAGEM_ERRO)

            VALUES ('PRC_CARGA_PAGAMENTO', USER, SYSDATE, '-20008',

                'Valor invalido (<= 0) informado para pagamento da consulta ' ||
                p_id_consulta);

        DBMS_OUTPUT.PUT_LINE('FALHA: valor invalido para pagamento da consulta ' ||
        p_id_consulta || '.');

    WHEN OTHERS THEN

        INSERT INTO LOG_ERRO_CARGA (NM_PROCEDURE, NM_USUARIO, DT_OCORRENCIA, CD_ERRO,
        DS_MENSAGEM_ERRO)

            VALUES ('PRC_CARGA_PAGAMENTO', USER, SYSDATE, SQLCODE, SUBSTR(SQLERRM, 1, 480));

        DBMS_OUTPUT.PUT_LINE('FALHA inesperada ao cadastrar pagamento: ' || SQLERRM);

    END PRC_CARGA_PAGAMENTO;

/

```

Regras de negocio validadas: existencia previa da consulta (FK) e valor estritamente positivo. Excecoes tratadas: e\_consulta\_inexiste, e\_valor\_invalido e OTHERS.

## 5.7 Demonstracao de uso (chamadas por passagem de parametro)

Cada procedure foi chamada por passagem de parametro — nunca com valores fixados dentro do corpo da procedure — em dois cenarios: um caso valido (INSERT efetivado) e um caso invalido (execucao tratada e registrada em LOG\_ERRO\_CARGA), evidenciando o funcionamento completo do tratamento de erros.

```

BEGIN PRC_CARGA_TUTOR('Roberta Nascimento', '123.456.789-01', '(11) 90000-1001',

    'roberta.nascimento@email.com', 'Rua Nova, 10 - Sao Paulo/SP'); END;

/

BEGIN PRC_CARGA_TUTOR('Tutor CPF Invalido', '123', '(11) 90000-1002',

```

```

'invalido@email.com', 'Rua Teste, 20'); END;

/

-- (chamadas analogas para PRC_CARGA_VETERINARIO, PRC_CARGA_PET,

-- PRC_CARGA_CONSULTA e PRC_CARGA_PAGAMENTO – ver Secao 12 do

-- arquivo 2TDSPO_2024_CodigoSql_Integrantes.sql)

SELECT ID_LOG_ERRO, NM_PROCEDURE, NM_USUARIO, DT_OCORRENCIA, CD_ERRO, DS_MENSAGEM_ERRO

FROM LOG_ERRO_CARGA

ORDER BY ID_LOG_ERRO;

```

### *Evidencia de execucao (print de tela)*

Saida real obtida ao consultar 'SELECT \* FROM LOG\_ERRO\_CARGA ORDER BY ID\_LOG\_ERRO;' apos rodar o script em mais de uma sessao. As linhas 1 a 5 mostram as excecoes customizadas de regra de negocio (uma por procedure de carga); as linhas 6 em diante mostram tambem a execucao nativa DUP\_VAL\_ON\_INDEX (ORA-00001) disparada quando o CPF/CRMV ja estava cadastrado em uma execucao anterior do script — evidenciando dois tipos distintos de execucao tratada por procedure:

| ID_LOG_ERRO | NM_PROCEDURE          | NM_USUARIO | DT_OCORRENCIA | CD_ERRO | DS_MENSAGEM_ERRO                                                                     |
|-------------|-----------------------|------------|---------------|---------|--------------------------------------------------------------------------------------|
| 1           | PRC_CARGA_TUTOR       | RM562396   | 12/09/26      | -20001  | CPF invalido ou nao informado para o tutor Tutor CPF Invalido                        |
| 2           | PRC_CARGA_VETERINARIO | RM562396   | 12/09/26      | -20002  | CRMV invalido ou nao informado para o veterinario Veterinario CRMV Invalido          |
| 3           | PRC_CARGA_PET         | RM562396   | 12/09/26      | -20003  | ID_TUTOR 9999 nao existe para o pet Pet Tutor Inexistente                            |
| 4           | PRC_CARGA_CONSULTA    | RM562396   | 12/09/26      | -20006  | Status invalido informado: EM_ANDAMENTO                                              |
| 5           | PRC_CARGA_PAGAMENTO   | RM562396   | 12/09/26      | -20008  | Valor invalido (<= 0) informado para pagamento da consulta 1                         |
| 6           | PRC_CARGA_TUTOR       | RM562396   | 12/09/26      | -1      | CPF ja cadastrado: ORA-00001: restricao exclusiva (RM562396.KV_UQ_TUTOR_CPF) violada |
| 7           | PRC_CARGA_TUTOR       | RM562396   | 12/09/26      | -20001  | CPF invalido ou nao informado para o tutor Tutor CPF Invalido                        |
| 8           | PRC_CARGA_VETERINARIO | RM562396   | 12/09/26      | -1      | CRMV ja cadastrado: ORA-00001: restricao exclusiva (RM562396.KV_UQ_VET_CRMV) violada |
| 9           | PRC_CARGA_VETERINARIO | RM562396   | 12/09/26      | -20002  | CRMV invalido ou nao informado para o veterinario Veterinario CRMV Invalido          |
| 10          | PRC_CARGA_PET         | RM562396   | 12/09/26      | -20003  | ID_TUTOR 9999 nao existe para o pet Pet Tutor Inexistente                            |
| 11          | PRC_CARGA_CONSULTA    | RM562396   | 12/09/26      | -20006  | Status invalido informado: EM_ANDAMENTO                                              |
| 12          | PRC_CARGA_PAGAMENTO   | RM562396   | 12/09/26      | -20008  | Valor invalido (<= 0) informado para pagamento da consulta 1                         |

Figura 3 — Conteudo da tabela LOG\_ERRO\_CARGA apos multiplas execucoes das procedures de carga

Fonte: elaborado pelos autores (execucao no Oracle Database, 2026).

Essa evidencia confirma que todas as 5 procedures de carga (PRC\_CARGA\_TUTOR, PRC\_CARGA\_VETERINARIO, PRC\_CARGA\_PET, PRC\_CARGA\_CONSULTA, PRC\_CARGA\_PAGAMENTO) capturam corretamente tanto as excecoes de regra de negocio customizadas quanto as excecoes nativas do Oracle, registrando o nome da procedure, o usuario, a data/hora, o codigo e a mensagem de erro em todos os casos.

## 6 CONSULTAS COM JUNCOES, AGRUPAMENTO E ORDENACAO

Esta secao apresenta 2 blocos anonicos contendo, no total, 3 consultas com juncao (JOIN) entre tabelas relacionais, cada uma utilizando agrupamento (GROUP BY) e ordenacao (ORDER BY), conforme exigido.

### 6.1 Bloco Anonimo 1 — engajamento por tutor/veterinario e faturamento por especie/clinica

Este bloco contem 2 consultas: a primeira junta TUTOR, PET, CONSULTA e VETERINARIO para contar consultas por tutor e veterinario; a segunda junta PET, CONSULTA e FATO\_PAGAMENTO para somar o faturamento por especie do pet e clinica.

```
DECLARE
```

```
BEGIN
```

```
    DBMS_OUTPUT.PUT_LINE('=== Quantidade de consultas por Tutor e Veterinario ===');
```

```
    FOR r IN (
```

```
        SELECT t.nome AS tutor, v.nome AS veterinario, COUNT(c.id_consulta) AS
qtd_consultas
```

```
        FROM    TUTOR t
```

```
        JOIN    PET p          ON p.id_tutor = t.id_tutor
```

```
        JOIN    CONSULTA c     ON c.id_pet = p.id_pet
```

```
        JOIN    VETERINARIO v  ON v.id_veterinario = c.id_veterinario
```

```
        GROUP BY t.nome, v.nome
```

```
        ORDER BY qtd_consultas DESC, t.nome ASC
```

```
    ) LOOP
```

```
        DBMS_OUTPUT.PUT_LINE(RPAD(r.tutor, 26) || RPAD(r.veterinario, 24) ||
r.qtd_consultas);
```

```
    END LOOP;
```

```
    DBMS_OUTPUT.PUT_LINE(' ');
```

```
    DBMS_OUTPUT.PUT_LINE('=== Faturamento total por Especie do Pet e Clinica ===');
```

```

FOR r IN (

    SELECT p.especie, f.clinica, SUM(f.valor) AS total_faturado

    FROM    PET p

    JOIN    CONSULTA c          ON c.id_pet = p.id_pet

    JOIN    FATO_PAGAMENTO f    ON f.id_consulta = c.id_consulta

    GROUP BY p.especie, f.clinica

    ORDER BY total_faturado DESC

) LOOP

    DBMS_OUTPUT.PUT_LINE(RPAD(r.especie, 12) || RPAD(r.clinica, 22) ||

                          'R$ ' || TO_CHAR(r.total_faturado, 'FM999G999D00'));

END LOOP;

END;

/

```

## 6.2 Bloco Anonimo 2 — diagnosticos por veterinario e tipo de consulta

Este bloco contem a terceira consulta com juncao, unindo VETERINARIO, CONSULTA, PET e TUTOR, agrupando por veterinario e tipo de consulta, e ordenando pelo nome do veterinario e pela quantidade.

```

DECLARE
BEGIN
    DBMS_OUTPUT.PUT_LINE('=== Quantidade de consultas por Veterinario e Tipo de Consulta
===');
    FOR r IN (
        SELECT v.nome AS veterinario, c.tipo_consulta, COUNT(*) AS qtd
        FROM    VETERINARIO v
        JOIN    CONSULTA c    ON c.id_veterinario = v.id_veterinario
        JOIN    PET p         ON p.id_pet = c.id_pet
        JOIN    TUTOR t       ON t.id_tutor = p.id_tutor
        GROUP BY v.nome, c.tipo_consulta
        ORDER BY v.nome ASC, qtd DESC
    ) LOOP
        DBMS_OUTPUT.PUT_LINE(RPAD(r.veterinario, 24) || RPAD(r.tipo_consulta, 18) ||
r.qtd);
    END LOOP;
END;

/

```

**Print de execucao pendente de anexacao**

Executar os blocos das Secoes 6.1 e 6.2 (Secao 13 do arquivo .sql) no Oracle e inserir os prints de tela das tres listagens geradas, mantendo os mesmos dados de carga documentados na Secao 2 para reprodutibilidade.

## 7 BLOCO: VALOR ATUAL, ANTERIOR E PROXIMO

Este bloco anonimo le a tabela FATO\_PAGAMENTO ordenada por ID\_PAGAMENTO e, para a coluna VALOR, exibe em cada linha o valor da linha atual, o valor da linha anterior e o valor da proxima linha. Quando nao existe linha anterior (primeira linha) ou proxima (ultima linha), o bloco exibe a palavra 'Vazio', conforme exigido. A tabela FATO\_PAGAMENTO possui 10 linhas (Secao 2), superando o minimo de 5 linhas solicitado.

### 7.1 Estrategia de implementacao

Os valores sao primeiro lidos para uma colecao em memoria (PL/SQL TABLE indexada por PLS\_INTEGER) percorrendo o cursor implicito uma unica vez. Em seguida, um segundo laco percorre essa colecao por indice, permitindo acessar diretamente a posicao anterior (i-1) e a posicao seguinte (i+1) de forma manual, sem uso de funcoes analiticas prontas como LAG/LEAD.

```
DECLARE
```

```
    TYPE t_valores IS TABLE OF FATO_PAGAMENTO.VALOR%TYPE INDEX BY PLS_INTEGER;
```

```
    v_valores      t_valores;
```

```
    v_qtd          PLS_INTEGER := 0;
```

```
    v_atual        VARCHAR2(20);
```

```
    v_anterior     VARCHAR2(20);
```

```
    v_proximo      VARCHAR2(20);
```

```
BEGIN
```

```
    FOR r IN (SELECT valor FROM FATO_PAGAMENTO ORDER BY id_pagamento) LOOP
```

```
        v_qtd := v_qtd + 1;
```

```
        v_valores(v_qtd) := r.valor;
```

```
    END LOOP;
```

```
    IF v_qtd = 0 THEN
```

```
        RAISE NO_DATA_FOUND;
```

```
    END IF;
```

```
DBMS_OUTPUT.PUT_LINE(RPAD('LINHA', 8) || RPAD('ANTERIOR', 15) || RPAD('ATUAL', 15) ||
'PROXIMO');
```

```
DBMS_OUTPUT.PUT_LINE(RPAD('-', 50, '-'));
```

```
FOR i IN 1..v_qtd LOOP
```

```
    v_atual := TO_CHAR(v_valores(i), 'FM999G999D00');
```

```
    IF i = 1 THEN
```

```
        v_anterior := 'Vazio';
```

```
    ELSE
```

```
        v_anterior := TO_CHAR(v_valores(i - 1), 'FM999G999D00');
```

```
    END IF;
```

```
    IF i = v_qtd THEN
```

```
        v_proximo := 'Vazio';
```

```
    ELSE
```

```
        v_proximo := TO_CHAR(v_valores(i + 1), 'FM999G999D00');
```

```
    END IF;
```

```
    DBMS_OUTPUT.PUT_LINE(RPAD(i, 8) || RPAD(v_anterior, 15) || RPAD(v_atual, 15) ||
v_proximo);
```

```
END LOOP;
```

```
EXCEPTION
```

```
    WHEN NO_DATA_FOUND THEN
```

```
        DBMS_OUTPUT.PUT_LINE('Aviso: a tabela FATO_PAGAMENTO nao possui registros.');
```

```

WHEN VALUE_ERROR THEN

    DBMS_OUTPUT.PUT_LINE('Erro: valor incompativel encontrado na coluna VALOR.');
```

```

WHEN OTHERS THEN

    DBMS_OUTPUT.PUT_LINE('Erro inesperado: ' || SQLERRM);

END;

/
```

## 7.2 Tratamento de excecoes

- NO\_DATA\_FOUND — disparada manualmente quando a tabela FATO\_PAGAMENTO nao possui linhas.
- VALUE\_ERROR — protecao contra valores incompativeis de tipo durante a conversao TO\_CHAR.
- OTHERS — captura generica de qualquer falha nao mapeada, exibindo o SQLERRM.

### Print de execucao pendente de anexacao

Executar este bloco (Secao 14 do arquivo .sql) no Oracle e inserir o print de tela com as 10 linhas geradas, evidenciando 'Vazio' na coluna ANTERIOR da primeira linha e na coluna PROXIMO da ultima linha.

## 8 RELATORIOS COM CURSOR EXPLICITO E TOMADA DE DECISAO

Esta secao apresenta 4 blocos anonicos que utilizam cursor explicito (OPEN/FETCH/CLOSE) combinado com tomada de decisao (IF/ELSIF). O ultimo bloco (Secao 8.4) e o relatorio completo: lista todos os dados de FATO\_PAGAMENTO, mostra os dados numericos sumarizados (subtotal por clinica) e apresenta a sumarizacao agrupada pelo criterio definido pelo grupo (CLINICA), seguindo o mesmo padrao de quebra manual de grupo e total geral adotado na Secao 9.3.

### 8.1 Relatorio 1 — classificacao etaria dos pets

DECLARE

```
CURSOR c_pet IS

    SELECT nome, especie, data_nascimento

    FROM    PET

    ORDER BY data_nascimento;

v_pet      c_pet%ROWTYPE;

v_meses    NUMBER;

v_categoria VARCHAR2(20);
```

BEGIN

```
DBMS_OUTPUT.PUT_LINE(RPAD('PET', 15) || RPAD('ESPECIE', 12) || 'CATEGORIA ETARIA');

DBMS_OUTPUT.PUT_LINE(RPAD('-', 45, '-'));


OPEN c_pet;

LOOP

    FETCH c_pet INTO v_pet;

    EXIT WHEN c_pet%NOTFOUND;


    v_meses := TRUNC(MONTHS_BETWEEN(SYSDATE, v_pet.data_nascimento));
```

```

        IF v_meses < 12 THEN

            v_categoria := 'Filhote';

        ELSIF v_meses < 84 THEN

            v_categoria := 'Adulto';

        ELSE

            v_categoria := 'Idoso';

        END IF;

        DBMS_OUTPUT.PUT_LINE(RPAD(v_pet.nome, 15) || RPAD(v_pet.especie, 12) ||
v_categoria);

    END LOOP;

    CLOSE c_pet;

EXCEPTION

    WHEN NO_DATA_FOUND THEN

        DBMS_OUTPUT.PUT_LINE('Aviso: nenhum pet cadastrado.');
```

```

    WHEN CURSOR_ALREADY_OPEN THEN

        DBMS_OUTPUT.PUT_LINE('Erro: cursor ja estava aberto.');
```

```

    WHEN OTHERS THEN

        IF c_pet%ISOPEN THEN CLOSE c_pet; END IF;

        DBMS_OUTPUT.PUT_LINE('Erro inesperado: ' || SQLERRM);

END;

/
```

## 8.2 Relatorio 2 — triagem de acao por status da consulta

Cursor explicito sobre CONSULTA que, por meio de decisao IF/ELSIF sobre o STATUS, recomenda uma acao operacional para a equipe da clinica.

```

DECLARE
```

```

CURSOR c_cons IS

    SELECT tipo_consulta, status

    FROM    CONSULTA

    ORDER BY status;

v_c      c_cons%ROWTYPE;

v_acao   VARCHAR2(40);

BEGIN

    DBMS_OUTPUT.PUT_LINE(RPAD('TIPO DE CONSULTA', 22) || RPAD('STATUS', 14) || 'ACAO
RECOMENDADA');

    DBMS_OUTPUT.PUT_LINE(RPAD('-', 60, '-'));

    OPEN c_cons;

    LOOP

        FETCH c_cons INTO v_c;

        EXIT WHEN c_cons%NOTFOUND;

        IF v_c.status = 'AGENDADA' THEN

            v_acao := 'Enviar lembrete ao tutor';

        ELSIF v_c.status = 'CANCELADA' THEN

            v_acao := 'Verificar motivo do cancelamento';

        ELSE

            v_acao := 'Arquivar prontuario';

        END IF;

        DBMS_OUTPUT.PUT_LINE(RPAD(v_c.tipo_consulta, 22) || RPAD(v_c.status, 14) ||
v_acao);

    END LOOP;

```

```

CLOSE c_cons;

EXCEPTION

    WHEN NO_DATA_FOUND THEN

        DBMS_OUTPUT.PUT_LINE('Aviso: nenhuma consulta cadastrada.');
```

```

    WHEN TOO_MANY_ROWS THEN

        DBMS_OUTPUT.PUT_LINE('Erro: retorno inesperado de multiplas linhas.');
```

```

    WHEN OTHERS THEN

        IF c_cons%ISOPEN THEN CLOSE c_cons; END IF;

        DBMS_OUTPUT.PUT_LINE('Erro inesperado: ' || SQLERRM);

END;

/
```

Excecoes tratadas: NO\_DATA\_FOUND, TOO\_MANY\_ROWS e OTHERS.

### 8.3 Relatorio 3 — classificacao de fidelidade do tutor

Cursor explicito sobre TUTOR com subconsulta de contagem de pets, classificando cada tutor como Cliente Ouro (2+ pets), Cliente Prata (1 pet) ou Sem pets, por decisao IF/ELSIF.

```

DECLARE

    CURSOR c_tutor IS

        SELECT t.nome,

            (SELECT COUNT(*) FROM PET p WHERE p.id_tutor = t.id_tutor) AS qtd_pets

        FROM    TUTOR t

        ORDER BY t.nome;

    v_t          c_tutor%ROWTYPE;

    v_categoria  VARCHAR2(20);

BEGIN

    DBMS_OUTPUT.PUT_LINE(RPAD('TUTOR', 30) || RPAD('QTD PETS', 10) || 'CATEGORIA');
```

```
DBMS_OUTPUT.PUT_LINE(RPAD('-', 60, '-'));

OPEN c_tutor;

LOOP

    FETCH c_tutor INTO v_t;

    EXIT WHEN c_tutor%NOTFOUND;

    IF v_t.qtd_pets >= 2 THEN

        v_categoria := 'Cliente Ouro';

    ELSIF v_t.qtd_pets = 1 THEN

        v_categoria := 'Cliente Prata';

    ELSE

        v_categoria := 'Sem pets';

    END IF;

    DBMS_OUTPUT.PUT_LINE(RPAD(v_t.nome, 30) || RPAD(v_t.qtd_pets, 10) || v_categoria);

END LOOP;

CLOSE c_tutor;

EXCEPTION

    WHEN NO_DATA_FOUND THEN

        DBMS_OUTPUT.PUT_LINE('Aviso: nenhum tutor cadastrado.');
```

```
    WHEN INVALID_CURSOR THEN

        DBMS_OUTPUT.PUT_LINE('Erro: operacao invalida sobre o cursor.');
```

```
    WHEN OTHERS THEN

        IF c_tutor%ISOPEN THEN CLOSE c_tutor; END IF;
```

```

        DBMS_OUTPUT.PUT_LINE('Erro inesperado: ' || SQLERRM);

END;

/

```

Excecoes tratadas: NO\_DATA\_FOUND, INVALID\_CURSOR e OTHERS.

#### 8.4 Relatorio 4 (completo) — listagem, sumario e agrupamento de pagamentos

Este e o bloco que atende integralmente ao requisito de listar todos os dados de uma tabela, mostrar os dados numericos sumariados e apresentar a sumario agrupada por um criterio definido pelo grupo (CLINICA), com cursor explicito e decisao manual de quebra de grupo — sem ROLLUP, CUBE ou GROUPING SETS.

```

DECLARE
    CURSOR c_pag IS
        SELECT clinica, tipo_pagamento, id_pagamento, valor
        FROM   FATO_PAGAMENTO
        ORDER BY clinica, id_pagamento;
    r_pag          c_pag%ROWTYPE;
    v_clinica_ant  VARCHAR2(60) := NULL;
    v_subtotal_clinica NUMBER(10,2) := 0;
    v_total_geral  NUMBER(10,2) := 0;
    v_qtd_linhas   NUMBER := 0;
BEGIN
    DBMS_OUTPUT.PUT_LINE('=== RELATORIO COMPLETO DE PAGAMENTOS (listagem + sumario)
    ===');
    DBMS_OUTPUT.PUT_LINE(RPAD('CLINICA', 22) || RPAD('TIPO PAGAMENTO', 18) || RPAD('ID',
    8) || 'VALOR (R$)');
    DBMS_OUTPUT.PUT_LINE(RPAD('-', 60, '-'));

    OPEN c_pag;
    LOOP
        FETCH c_pag INTO r_pag;
        EXIT WHEN c_pag%NOTFOUND;

        v_qtd_linhas := v_qtd_linhas + 1;

        -- Tomada de decisao: deteccao manual de quebra de grupo (CLINICA)
        IF v_clinica_ant IS NOT NULL AND r_pag.clinica <> v_clinica_ant THEN
            DBMS_OUTPUT.PUT_LINE(RPAD('Subtotal ' || v_clinica_ant, 48) ||
            TO_CHAR(v_subtotal_clinica, 'FM999G999D00'));
            DBMS_OUTPUT.PUT_LINE(RPAD('-', 60, '-'));
            v_subtotal_clinica := 0;
        END IF;

        v_clinica_ant := r_pag.clinica;

        DBMS_OUTPUT.PUT_LINE(RPAD(r_pag.clinica, 22) || RPAD(r_pag.tipo_pagamento, 18)
        ||
        RPAD(r_pag.id_pagamento, 8) || TO_CHAR(r_pag.valor,
        'FM999G999D00'));

        v_subtotal_clinica := v_subtotal_clinica + r_pag.valor;
    
```

```

        v_total_geral      := v_total_geral + r_pag.valor;
    END LOOP;
    CLOSE c_pag;

    IF v_qtd_linhas = 0 THEN
        RAISE NO_DATA_FOUND;
    END IF;

    DBMS_OUTPUT.PUT_LINE(RPAD('Subtotal ' || v_clinica_ant, 48) ||
        TO_CHAR(v_subtotal_clinica, 'FM999G999D00'));
    DBMS_OUTPUT.PUT_LINE(RPAD('=', 60, '='));
    DBMS_OUTPUT.PUT_LINE(RPAD('TOTAL GERAL SUMARIZADO', 48) ||
        TO_CHAR(v_total_geral, 'FM999G999D00'));

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Aviso: nao ha pagamentos cadastrados.');
```

```

    WHEN ZERO_DIVIDE THEN
        DBMS_OUTPUT.PUT_LINE('Erro: divisao por zero detectada.');
```

```

    WHEN OTHERS THEN
        IF c_pag%ISOPEN THEN CLOSE c_pag; END IF;
        DBMS_OUTPUT.PUT_LINE('Erro inesperado: ' || SQLERRM);
```

```

END;
/
```

#### **Print de execucao pendente de anexacao**

Executar os 4 relatorios (Secao 15 do arquivo .sql) no Oracle e inserir os prints de tela nas Secoes 8.1 a 8.4, seguindo o mesmo padrao de evidencia usado nas Secoes 9 e 10 (imagem + legenda Figura N + Fonte).

## 9 FUNCOES E PROCEDIMENTOS DE APOIO

Esta secao consolida os objetos PL/SQL desenvolvidos e ja validados na Sprint anterior, mantidos nesta entrega por continuarem atendendo a requisitos vigentes do projeto (conversao de dados para JSON sem funcoes nativas, subtotais manuais e regra de negocio de idade do pet). Todas as evidencias de execucao abaixo sao prints reais, capturados em execucao no Oracle Database.

### 9.1 Funcao 1 — Conversor JSON Manual (FN\_MONTA\_JSON)

Recebe os dados ja relacionados (id da consulta, pet, tutor, veterinario, data e tipo) e devolve uma string no formato JSON, construida inteiramente por concatenacao de texto (operador ||). Nenhuma funcao nativa de conversao para JSON do Oracle (TO\_JSON, JSON\_OBJECT, JSON\_VALUE, JSON\_QUERY, JSON\_TABLE) e utilizada.

*Codigo-fonte comentado*

```
CREATE OR REPLACE FUNCTION FN_MONTA_JSON (

    p_id_consulta    IN NUMBER,

    p_pet            IN VARCHAR2,

    p_tutor          IN VARCHAR2,

    p_veterinario    IN VARCHAR2,

    p_data_consulta  IN VARCHAR2,

    p_tipo_consulta  IN VARCHAR2

) RETURN VARCHAR2
IS
    v_json           VARCHAR2(4000);
BEGIN
    -- Validacao basica: parametro chave nao pode ser nulo

    IF p_id_consulta IS NULL THEN

        RAISE VALUE_ERROR;

    END IF;
```

```

-- Montagem 100% manual da string JSON via concatenacao (||)

-- PROIBIDO usar TO_JSON, JSON_OBJECT, JSON_ARRAY, etc.

v_json := '{' ||

        '"id_consulta":' || TO_CHAR(p_id_consulta) || ',' ||

        '"pet":' || NVL(p_pet, 'N/A') || ',' ||

        '"tutor":' || NVL(p_tutor, 'N/A') || ',' ||

        '"veterinario":' || NVL(p_veterinario, 'N/A') || ',' ||

        '"data_consulta":' || NVL(p_data_consulta, 'N/A') || ',' ||

        '"tipo_consulta":' || NVL(p_tipo_consulta, 'N/A') || '' ||

        '}' ;

RETURN v_json;

EXCEPTION

    WHEN NO_DATA_FOUND THEN

        RETURN '{"erro": "Nenhum dado encontrado para montagem do JSON"}';

    WHEN TOO_MANY_ROWS THEN

        RETURN '{"erro": "Mais de um registro retornado para montagem do JSON"}';

    WHEN VALUE_ERROR THEN

        RETURN '{"erro": "Parametro obrigatorio invalido ou nulo"}';

    WHEN OTHERS THEN

        RETURN '{"erro": "Falha inesperada: ' || SQLERRM || '"}';

END FN_MONTA_JSON;

/

```

### *Tratamento de excecoes*

- **VALUE\_ERROR** — disparada manualmente quando o parametro obrigatorio `p_id_consulta` e nulo.

- NO\_DATA\_FOUND — protecao para o caso de a funcao ser usada futuramente com SELECT INTO sem retorno.
- TOO\_MANY\_ROWS — protecao para o caso de retorno inesperado de multiplas linhas.
- OTHERS — captura generica de qualquer erro nao previsto, retornando o SQLERRM dentro do proprio JSON de erro.

## 9.2 Procedimento 1 — JOIN entre Tabelas + JSON (PROC\_LISTAR\_CONSULTAS\_JSON)

Realiza um JOIN entre quatro tabelas relacionais (CONSULTA, PET, TUTOR e VETERINARIO), percorre o resultado com um cursor implicito (FOR loop) e, para cada linha, invoca a Funcao 1 para montar a saida em formato JSON, exibida via DBMS\_OUTPUT.

*Codigo-fonte comentado*

```
CREATE OR REPLACE PROCEDURE PROC_LISTAR_CONSULTAS_JSON
IS
    -- Cursor com JOIN entre 4 tabelas relacionais

    CURSOR c_consultas IS

        SELECT c.id_consulta,

               p.nome AS nome_pet,

               t.nome AS nome_tutor,

               v.nome AS nome_vet,

               TO_CHAR(c.data_consulta, 'DD/MM/YYYY') AS data_fmt,

               c.tipo_consulta

        FROM   CONSULTA c

        JOIN   PET p           ON p.id_pet = c.id_pet

        JOIN   TUTOR t         ON t.id_tutor = p.id_tutor

        JOIN   VETERINARIO v   ON v.id_veterinario = c.id_veterinario

        ORDER BY c.id_consulta;
```

```
v_json          VARCHAR2(4000);

v_total         NUMBER := 0;

BEGIN

    SELECT COUNT(*) INTO v_total FROM CONSULTA;

    IF v_total = 0 THEN

        RAISE NO_DATA_FOUND;

    END IF;

    DBMS_OUTPUT.PUT_LINE('=== LISTAGEM DE CONSULTAS (JSON MANUAL) ===');

    -- Para cada linha do JOIN, chama a Funcao 1 (FN_MONTA_JSON)

    FOR r_reg IN c_consultas LOOP

        v_json := FN_MONTA_JSON(

            p_id_consulta => r_reg.id_consulta,

            p_pet          => r_reg.nome_pet,

            p_tutor        => r_reg.nome_tutor,

            p_veterinario  => r_reg.nome_vet,

            p_data_consulta => r_reg.data_fmt,

            p_tipo_consulta => r_reg.tipo_consulta

        );

        DBMS_OUTPUT.PUT_LINE(v_json);

    END LOOP;

EXCEPTION

    WHEN NO_DATA_FOUND THEN
```

```

DBMS_OUTPUT.PUT_LINE('Aviso: Nenhuma consulta cadastrada no sistema.');
```

WHEN TOO\_MANY\_ROWS THEN

```

DBMS_OUTPUT.PUT_LINE('Erro: Retorno de dados excedeu o esperado.');
```

WHEN OTHERS THEN

```

DBMS_OUTPUT.PUT_LINE('Erro inesperado em PROC_LISTAR_CONSULTAS_JSON: ' || SQLERRM);
```

END PROC\_LISTAR\_CONSULTAS\_JSON;

/

### *Evidencia de execucao (print de tela)*

Saida real obtida ao executar 'BEGIN PROC\_LISTAR\_CONSULTAS\_JSON; END;' no Oracle. Alem das 10 consultas da carga inicial, aparece tambem a ID\_CONSULTA=100, incluida posteriormente pela procedure PRC\_CARGA\_CONSULTA (Secao 5.7), confirmando que a funcao de JSON manual continua funcionando corretamente mesmo apos a carga adicional da Sprint 3:

```

=== LISTAGEM DE CONSULTAS (JSON MANUAL) ===
{"id_consulta":1,"pet":"Thor","tutor":"Ana Beatriz Souza","veterinario":"Dra. Patricia Gomes","data_consulta":"10/01/2025","tipo_consulta":"Checkup"}
{"id_consulta":2,"pet":"Mel","tutor":"Carlos Eduardo Lima","veterinario":"Dr. Rafael Andrade","data_consulta":"14/02/2025","tipo_consulta":"Dermatologia"}
{"id_consulta":3,"pet":"Bidu","tutor":"Beatriz Fernandes","veterinario":"Dra. Camila Duarte","data_consulta":"05/03/2025","tipo_consulta":"Cardiologia"}
{"id_consulta":4,"pet":"Luna","tutor":"Diego Almeida","veterinario":"Dra. Patricia Gomes","data_consulta":"18/04/2025","tipo_consulta":"Vacinacao"}
{"id_consulta":5,"pet":"Rex","tutor":"Fernanda Costa","veterinario":"Dr. Bruno Cardoso","data_consulta":"22/05/2025","tipo_consulta":"Ortopedia"}
{"id_consulta":6,"pet":"Mina","tutor":"Gabriel Rocha","veterinario":"Dra. Larissa Ferreira","data_consulta":"30/06/2025","tipo_consulta":"Checkup"}
{"id_consulta":7,"pet":"Toby","tutor":"Helena Martins","veterinario":"Dr. Eduardo Santos","data_consulta":"11/07/2025","tipo_consulta":"Cirurgia"}
{"id_consulta":8,"pet":"Mimi","tutor":"Igor Nogueira","veterinario":"Dra. Renata Barbosa","data_consulta":"09/08/2025","tipo_consulta":"Oncologia"}
{"id_consulta":9,"pet":"Zeus","tutor":"Juliana Pires","veterinario":"Dr. Felipe Ramos","data_consulta":"25/09/2025","tipo_consulta":"Checkup"}
{"id_consulta":10,"pet":"Belinha","tutor":"Marcos Vinicius Teixeira","veterinario":"Dra. Vanessa Lopes","data_consulta":"15/01/2026","tipo_consulta":"Nutricao"}
{"id_consulta":100,"pet":"Thor","tutor":"Ana Beatriz Souza","veterinario":"Dra. Patricia Gomes","data_consulta":"12/09/2026","tipo_consulta":"Checkup"}

Procedimento PL/SQL concluido com sucesso.
```

Figura 4 — Saida do PROC\_LISTAR\_CONSULTAS\_JSON (JSON manual, 11 registros apos carga adicional)

Fonte: elaborado pelos autores (execucao no Oracle Database, 2026).

### *Tratamento de excecoes*

- NO\_DATA\_FOUND — disparada manualmente quando a contagem de consultas e zero.
- TOO\_MANY\_ROWS — protecao contra retorno inesperado do SELECT COUNT(\*).
- OTHERS — captura generica de qualquer falha nao mapeada, exibindo o SQLERRM.

## 9.3 Procedimento 2 — Subtotais Manuais (PROC\_SUBTOTAIS\_PAGAMENTO)

Le a tabela de fato FATO\_PAGAMENTO e calcula, de forma 100% manual (variaveis de controle e deteccao de quebra de grupo dentro do laco do cursor), os subtotais por CLINICA e o total geral — sem uso de ROLLUP, CUBE, GROUPING SETS ou GROUPING.

*Codigo-fonte comentado*

```

CREATE OR REPLACE PROCEDURE PROC_SUBTOTAIS_PAGAMENTO

IS

    CURSOR c_pag IS

        SELECT clinica, tipo_pagamento, id_pagamento, valor

        FROM    FATO_PAGAMENTO

        ORDER BY clinica, tipo_pagamento, id_pagamento;

    r_pag          c_pag%ROWTYPE;

    v_clinica_ant   VARCHAR2(60) := NULL;

    v_tipo_ant      VARCHAR2(30) := NULL;

    v_subtotal_tipo NUMBER(10,2) := 0;

    v_subtotal_clinica NUMBER(10,2) := 0;

    v_total_geral   NUMBER(10,2) := 0;

    v_qtd_linhas    NUMBER        := 0;

BEGIN

    OPEN c_pag;

    -- Cabecalho da tabela de saida

    DBMS_OUTPUT.PUT_LINE(RPAD('CLINICA', 25) || ' | ' ||

                           RPAD('TIPO PAGAMENTO', 20) || ' | ' || 'VALOR (R$)');

    DBMS_OUTPUT.PUT_LINE(RPAD('-', 25, '-') || '-|- ' ||

                           RPAD('-', 20, '-') || '-|- ' || RPAD('-', 15, '-'));

    LOOP

        FETCH c_pag INTO r_pag;

        EXIT WHEN c_pag%NOTFOUND;
    
```

```

v_qtd_linhas := v_qtd_linhas + 1;

-- QUEBRA MANUAL DE GRUPO (primeira categoria: CLINICA)
IF v_clinica_ant IS NOT NULL AND r_pag.clinica <> v_clinica_ant THEN

    DBMS_OUTPUT.PUT_LINE(RPAD('Sub Total', 25) || ' | ' ||

        RPAD(' ', 20) || ' | ' || TO_CHAR(v_subtotal_clinica, 'FM999G999D00'));

    DBMS_OUTPUT.PUT_LINE(RPAD('-', 66, '-'));

    v_subtotal_clinica := 0;

    v_subtotal_tipo    := 0;

    v_tipo_ant         := NULL;

END IF;

v_clinica_ant := r_pag.clinica;

v_tipo_ant    := r_pag.tipo_pagamento;

-- Imprime a linha detalhada atual

DBMS_OUTPUT.PUT_LINE(RPAD(r_pag.clinica, 25) || ' | ' ||

    RPAD(r_pag.tipo_pagamento, 20) || ' | ' ||

    TO_CHAR(r_pag.valor, 'FM999G999D00'));

-- ACUMULO MANUAL DOS TOTAIS (sem ROLLUP/CUBE/GROUPING SETS)

v_subtotal_tipo    := v_subtotal_tipo + r_pag.valor;

v_subtotal_clinica := v_subtotal_clinica + r_pag.valor;

v_total_geral      := v_total_geral + r_pag.valor;

END LOOP;

```

```

CLOSE c_pag;

IF v_qtd_linhas = 0 THEN

    RAISE NO_DATA_FOUND;

END IF;

-- Fecha o ultimo grupo pendente e imprime o total geral

IF v_clinica_ant IS NOT NULL THEN

    DBMS_OUTPUT.PUT_LINE(RPAD('Sub Total', 25) || ' | ' ||

        RPAD(' ', 20) || ' | ' || TO_CHAR(v_subtotal_clinica, 'FM999G999D00'));

END IF;

DBMS_OUTPUT.PUT_LINE(RPAD('=', 66, '='));

DBMS_OUTPUT.PUT_LINE(RPAD('Total Geral', 25) || ' | ' ||

    RPAD(' ', 20) || ' | ' || TO_CHAR(v_total_geral, 'FM999G999D00'));

EXCEPTION

    WHEN NO_DATA_FOUND THEN

        DBMS_OUTPUT.PUT_LINE('Aviso: Nao ha registros na tabela FATO_PAGAMENTO.');
```

```

    WHEN TOO_MANY_ROWS THEN

        DBMS_OUTPUT.PUT_LINE('Erro: Consulta retornou mais linhas do que o esperado.');
```

```

    WHEN ZERO_DIVIDE THEN

        DBMS_OUTPUT.PUT_LINE('Erro: Divisao por zero detectada.');
```

```

    WHEN OTHERS THEN

        IF c_pag%ISOPEN THEN

```

```

CLOSE c_pag;

END IF;

DBMS_OUTPUT.PUT_LINE('Erro inesperado: ' || SQLERRM);

END PROC_SUBTOTAIS_PAGAMENTO;

/

```

### *Evidencia de execucao (print de tela)*

Saida real obtida ao executar 'BEGIN PROC\_SUBTOTAIS\_PAGAMENTO; END;' apos a carga adicional da Sprint 3. Observa-se o subtotal por clinica (KuraVet Alphaville: R\$ 680,00; KuraVet Moema: R\$ 770,00; KuraVet Pinheiros: R\$ 725,00, ja incluindo o pagamento de R\$ 175,00 inserido via PRC\_CARGA\_PAGAMENTO) e o total geral de R\$ 2.175,00:

| CLINICA            | TIPO PAGAMENTO    | VALOR (R\$) |
|--------------------|-------------------|-------------|
| KuraVet Alphaville | Cartao de Credito | 310,00      |
| KuraVet Alphaville | Dinheiro          | 95,00       |
| KuraVet Alphaville | Pix               | 275,00      |
| Sub Total          |                   | 680,00      |
| KuraVet Moema      | Cartao de Credito | 350,00      |
| KuraVet Moema      | Cartao de Credito | 130,00      |
| KuraVet Moema      | Dinheiro          | 200,00      |
| KuraVet Moema      | Pix               | 90,00       |
| Sub Total          |                   | 770,00      |
| KuraVet Pinheiros  | Cartao de Credito | 220,00      |
| KuraVet Pinheiros  | Pix               | 150,00      |
| KuraVet Pinheiros  | Pix               | 180,00      |
| KuraVet Pinheiros  | Pix               | 175,00      |
| Sub Total          |                   | 725,00      |
| Total Geral        |                   | 2.175,00    |

Procedimento PL/SQL concluido com sucesso.

Figura 5 — Saida do PROC\_SUBTOTAIS\_PAGAMENTO com subtotais por clinica e total geral, apos carga adicional

Fonte: elaborado pelos autores (execucao no Oracle Database, 2026).

### *Tratamento de excecoes*

- NO\_DATA\_FOUND — disparada manualmente quando o cursor nao retorna nenhuma linha.

- `TOO_MANY_ROWS` — protecao adicional para operacoes de leitura single-row eventualmente adicionadas ao fluxo.
- `ZERO_DIVIDE` — protecao para eventual evolucao do calculo que envolva divisao (ex.: media por grupo).
- `OTHERS` — captura generica, fechando o cursor se ainda estiver aberto (`c_pag%ISOPEN`) antes de reportar o erro via `SQLERRM`.

#### 9.4 Funcao 2 — Regra de Negocio: Idade do Pet (FN\_CALCULA\_IDADE\_PET)

Calcula a idade exata de um pet, em anos e meses, a partir da data de nascimento cadastrada — usada para orientar o cronograma de saude preventiva e para exibicao direta ao tutor.

*Codigo-fonte comentado*

```
CREATE OR REPLACE FUNCTION FN_CALCULA_IDADE_PET (

    p_id_pet IN NUMBER

) RETURN VARCHAR2

IS

    v_data_nasc      PET.DATA_NASCIMENTO%TYPE;

    v_meses_total    NUMBER;

    v_anos           NUMBER;

    v_meses          NUMBER;

    v_resultado      VARCHAR2(200);

BEGIN

    IF p_id_pet IS NULL THEN

        RAISE VALUE_ERROR;

    END IF;

    -- Busca a data de nascimento do pet informado

    SELECT data_nascimento
```

```

INTO    v_data_nasc

FROM    PET

WHERE   id_pet = p_id_pet;

-- Regra de negocio: idade exata em meses completos

v_meses_total := TRUNC(MONTHS_BETWEEN(SYSDATE, v_data_nasc));

IF v_meses_total < 0 THEN

    RAISE VALUE_ERROR; -- data de nascimento futura e invalida

END IF;

v_anos := TRUNC(v_meses_total / 12);

v_meses := MOD(v_meses_total, 12);

IF v_meses_total < 1 THEN

    v_resultado := 'Filhote recém-nascido (menos de 1 mes)';

ELSE

    v_resultado := v_anos || ' ano(s) e ' || v_meses || ' mes(es)';

END IF;

RETURN v_resultado;

EXCEPTION

    WHEN NO_DATA_FOUND THEN

        RETURN 'Erro: Pet ID ' || p_id_pet || ' nao encontrado.';

    WHEN TOO_MANY_ROWS THEN

```

```

        RETURN 'Erro: Mais de um pet encontrado para o mesmo ID.';

    WHEN VALUE_ERROR THEN

        RETURN 'Erro: Parametro invalido ou data de nascimento inconsistente.';

    WHEN OTHERS THEN

        RETURN 'Erro inesperado: ' || SQLERRM;

END FN_CALCULA_IDADE_PET;

/

```

### *Evidencia de execucao (print de tela)*

Saida real obtida ao chamar a funcao para o Pet ID 1 (existente), Pet ID 10 (existente) e Pet ID 999 (inexistente), demonstrando tanto o calculo correto quanto o tratamento de execucao NO\_DATA\_FOUND:

```

Idade do Pet ID 1: 4 ano(s) e 5 mes(es)
Idade do Pet ID 10: 2 ano(s) e 6 mes(es)
Idade do Pet ID 999 (inexistente): Erro: Pet ID 999 nao encontrado.

Procedimento PL/SQL concluído com sucesso.

```

Figura 6 — Saida da FN\_CALCULA\_IDADE\_PET, incluindo o caso de erro tratado (Pet ID 999)

Fonte: elaborado pelos autores (execucao no Oracle Database, 2026).

### *Tratamento de excecoes*

- VALUE\_ERROR — disparada quando o parametro de entrada e nulo ou quando a data de nascimento resulta em idade negativa (data futura).
- NO\_DATA\_FOUND — disparada automaticamente pelo SELECT INTO quando o ID do pet nao existe na tabela PET (evidenciado no print acima, para o ID 999).
- TOO\_MANY\_ROWS — protecao contra a hipotese de duplicidade de ID\_PET (violacao teorica da PK).
- OTHERS — captura generica de qualquer erro nao mapeado, retornando o SQLERRM.

## 10 TRIGGER DE AUDITORIA (TRG\_AUDITORIA\_CONSULTA)

Para atender ao requisito de auditoria DML, foi criada a tabela AUDITORIA\_LOG (Secao 2.7) e a trigger TRG\_AUDITORIA\_CONSULTA, do tipo AFTER INSERT OR UPDATE OR DELETE, associada a tabela CONSULTA e disparada FOR EACH ROW. A trigger identifica o tipo de operacao (INSERTING / UPDATING / DELETING), monta os valores antigos e novos concatenando as colunas relevantes e insere um registro de log contendo usuario (USER), operacao, data/hora (SYSDATE), tabela afetada e os valores :OLD e :NEW.

### 10.1 Codigo-fonte comentado

```
CREATE OR REPLACE TRIGGER TRG_AUDITORIA_CONSULTA

AFTER INSERT OR UPDATE OR DELETE ON CONSULTA

FOR EACH ROW

DECLARE

    v_operacao  VARCHAR2(10);

    v_old_vals  VARCHAR2(4000);

    v_new_vals  VARCHAR2(4000);

BEGIN

    IF INSERTING THEN

        v_operacao := 'INSERT';

        v_old_vals := NULL;

        v_new_vals := 'ID_CONSULTA=' || :NEW.id_consulta ||

            ';ID_PET=' || :NEW.id_pet ||

            ';ID_VETERINARIO=' || :NEW.id_veterinario ||

            ';DATA_CONSULTA=' || TO_CHAR(:NEW.data_consulta,'DD/MM/YYYY') ||

            ';TIPO_CONSULTA=' || :NEW.tipo_consulta ||

            ';STATUS=' || :NEW.status;

    ELSIF UPDATING THEN
```

```

v_operacao := 'UPDATE';

v_old_vals := 'ID_CONSULTA=' || :OLD.id_consulta ||

              ';ID_PET=' || :OLD.id_pet ||

              ';ID_VETERINARIO=' || :OLD.id_veterinario ||

              ';DATA_CONSULTA=' || TO_CHAR(:OLD.data_consulta,'DD/MM/YYYY') ||

              ';TIPO_CONSULTA=' || :OLD.tipo_consulta ||

              ';STATUS=' || :OLD.status;

v_new_vals := 'ID_CONSULTA=' || :NEW.id_consulta ||

              ';ID_PET=' || :NEW.id_pet ||

              ';ID_VETERINARIO=' || :NEW.id_veterinario ||

              ';DATA_CONSULTA=' || TO_CHAR(:NEW.data_consulta,'DD/MM/YYYY') ||

              ';TIPO_CONSULTA=' || :NEW.tipo_consulta ||

              ';STATUS=' || :NEW.status;

ELSIF DELETING THEN

v_operacao := 'DELETE';

v_old_vals := 'ID_CONSULTA=' || :OLD.id_consulta ||

              ';ID_PET=' || :OLD.id_pet ||

              ';ID_VETERINARIO=' || :OLD.id_veterinario ||

              ';DATA_CONSULTA=' || TO_CHAR(:OLD.data_consulta,'DD/MM/YYYY') ||

              ';TIPO_CONSULTA=' || :OLD.tipo_consulta ||

              ';STATUS=' || :OLD.status;

v_new_vals := NULL;

END IF;

-- Grava o log com usuario Oracle, operacao, data/hora e valores OLD/NEW

INSERT INTO AUDITORIA_LOG (USUARIO, OPERACAO, DATA_HORA, TABELA_AFETADA,

```

```

        VALORES_OLD, VALORES_NEW)

VALUES (USER, v_operacao, SYSDATE, 'CONSULTA', v_old_vals, v_new_vals);

END TRG_AUDITORIA_CONSULTA;

/

```

## 10.2 Evidencia de execucao (print de tela)

Teste realizado com dois ciclos completos de INSERT -> UPDATE -> DELETE na CONSULTA de ID 11, alem de um INSERT adicional feito pela procedure PRC\_CARGA\_CONSULTA (ID\_CONSULTA=100, Secao 5.7). A consulta na tabela AUDITORIA\_LOG confirma os 7 registros gerados automaticamente pela trigger, com o usuario RM562396, a operacao correta, a data/hora do disparo e os valores OLD/NEW capturados a cada etapa. Por conta da largura da tabela, a evidencia foi capturada em duas imagens: a primeira com as colunas ID\_LOG a VALORES\_OLD, e a segunda com a coluna VALORES\_NEW correspondente a cada linha:

| ID_LOG | USUARIO  | OPERACAO | TABELA_AFETADA | DATA_HORA | VALORES_OLD                                                                                              |
|--------|----------|----------|----------------|-----------|----------------------------------------------------------------------------------------------------------|
| 1      | RM562396 | INSERT   | CONSULTA       | 12/09/26  | (null)                                                                                                   |
| 2      | RM562396 | UPDATE   | CONSULTA       | 12/09/26  | ID_CONSULTA=11;ID_PET=1;ID_VETERINARIO=2;DATA_CONSULTA=12/09/2026;TIPO_CONSULTA=Retorno;STATUS=AGENDADA  |
| 3      | RM562396 | DELETE   | CONSULTA       | 12/09/26  | ID_CONSULTA=11;ID_PET=1;ID_VETERINARIO=2;DATA_CONSULTA=12/09/2026;TIPO_CONSULTA=Retorno;STATUS=REALIZADA |
| 4      | RM562396 | INSERT   | CONSULTA       | 12/09/26  | (null)                                                                                                   |
| 5      | RM562396 | INSERT   | CONSULTA       | 12/09/26  | (null)                                                                                                   |
| 6      | RM562396 | UPDATE   | CONSULTA       | 12/09/26  | ID_CONSULTA=11;ID_PET=1;ID_VETERINARIO=2;DATA_CONSULTA=12/09/2026;TIPO_CONSULTA=Retorno;STATUS=AGENDADA  |
| 7      | RM562396 | DELETE   | CONSULTA       | 12/09/26  | ID_CONSULTA=11;ID_PET=1;ID_VETERINARIO=2;DATA_CONSULTA=12/09/2026;TIPO_CONSULTA=Retorno;STATUS=REALIZADA |

Figura 7 — Tabela AUDITORIA\_LOG — colunas ID\_LOG, USUARIO, OPERACAO, TABELA\_AFETADA, DATA\_HORA e VALORES\_OLD

Fonte: elaborado pelos autores (execucao no Oracle Database, 2026).

| VALORES_NEW                                                                                              |
|----------------------------------------------------------------------------------------------------------|
| ID_CONSULTA=11;ID_PET=1;ID_VETERINARIO=2;DATA_CONSULTA=12/09/2026;TIPO_CONSULTA=Retorno;STATUS=AGENDADA  |
| ID_CONSULTA=11;ID_PET=1;ID_VETERINARIO=2;DATA_CONSULTA=12/09/2026;TIPO_CONSULTA=Retorno;STATUS=REALIZADA |
| (null)                                                                                                   |
| ID_CONSULTA=100;ID_PET=1;ID_VETERINARIO=1;DATA_CONSULTA=12/09/2026;TIPO_CONSULTA=Checkup;STATUS=AGENDADA |
| ID_CONSULTA=11;ID_PET=1;ID_VETERINARIO=2;DATA_CONSULTA=12/09/2026;TIPO_CONSULTA=Retorno;STATUS=AGENDADA  |
| ID_CONSULTA=11;ID_PET=1;ID_VETERINARIO=2;DATA_CONSULTA=12/09/2026;TIPO_CONSULTA=Retorno;STATUS=REALIZADA |
| (null)                                                                                                   |

Figura 8 — Tabela AUDITORIA\_LOG — coluna VALORES\_NEW correspondente a cada linha da imagem anterior

Fonte: elaborado pelos autores (execucao no Oracle Database, 2026).

## 10.3 Analise do resultado

- Linhas 1-3 (primeiro ciclo): INSERT com VALORES\_OLD nulo; UPDATE de STATUS=AGENDADA para STATUS=REALIZADA; DELETE com VALORES\_NEW nulo, confirmando a exclusao do registro.

- Linha 4: INSERT da ID\_CONSULTA=100, originado pela procedure PRC\_CARGA\_CONSULTA (Secao 5.7) — evidencia que a trigger audita corretamente tambem as insercoes feitas por procedure, nao apenas DML manual.
- Linhas 5-7 (segundo ciclo): repeticao do ciclo INSERT -> UPDATE -> DELETE reaproveitando o ID\_CONSULTA=11 (livre apos o DELETE da linha 3), confirmando o funcionamento consistente da trigger em execucoes repetidas.

## 11 TRATAMENTO DE EXCECOES — VISAO CONSOLIDADA

Conforme exigido no feedback do professor, todos os blocos nomeados (functions e procedures) e todos os blocos anônimos deste projeto possuem no mínimo 3 cláusulas EXCEPTION WHEN distintas, sempre incluindo WHEN OTHERS. A tabela abaixo consolida essa cobertura para a totalidade dos objetos desenvolvidos nesta e na Sprint anterior:

Quadro 8 — Cobertura de tratamento de exceções por objeto PL/SQL

| Objeto                             | Exceções tratadas                                 | Qtd. |
|------------------------------------|---------------------------------------------------|------|
| FN_MONTA_JSON                      | NO_DATA_FOUND, TOO_MANY_ROWS, VALUE_ERROR, OTHERS | 4    |
| PROC_LISTAR_CONSULTAS_JSON         | NO_DATA_FOUND, TOO_MANY_ROWS, OTHERS              | 3    |
| PROC_SUBTOTAIS_PAGAMENTO           | NO_DATA_FOUND, TOO_MANY_ROWS, ZERO_DIVIDE, OTHERS | 4    |
| FN_CALCULA_IDADE_PET               | NO_DATA_FOUND, TOO_MANY_ROWS, VALUE_ERROR, OTHERS | 4    |
| PRC_CARGA_TUTOR                    | e_cpf_invalido, DUP_VAL_ON_INDEX, OTHERS          | 3    |
| PRC_CARGA_VETERINARIO              | e_crmv_invalido, DUP_VAL_ON_INDEX, OTHERS         | 3    |
| PRC_CARGA_PET                      | e_tutor_inexistente, e_data_nasc_futura, OTHERS   | 3    |
| PRC_CARGA_CONSULTA                 | e_pet_inexistente, e_status_invalido, OTHERS      | 3    |
| PRC_CARGA_PAGAMENTO                | e_consulta_inexiste, e_valor_invalido, OTHERS     | 3    |
| Bloco Valor Atual/Anterior/Proximo | NO_DATA_FOUND, VALUE_ERROR, OTHERS                | 3    |
| Relatorio 1 (classificacao etaria) | NO_DATA_FOUND, CURSOR_ALREADY_OPEN, OTHERS        | 3    |
| Relatorio 2 (triagem de status)    | NO_DATA_FOUND, TOO_MANY_ROWS, OTHERS              | 3    |
| Relatorio 3 (fidelidade do tutor)  | NO_DATA_FOUND, INVALID_CURSOR, OTHERS             | 3    |
| Relatorio 4 (completo, resumido)   | NO_DATA_FOUND, ZERO_DIVIDE, OTHERS                | 3    |

Fonte: elaborado pelos autores a partir do script\_bd.sql e do 2TDSPO\_2024\_CodigoSql\_Integrantes.sql (2026).

Adicionalmente, todos os blocos de limpeza idempotente do script (DROP ... EXCEPTION WHEN OTHERS THEN NULL) garantem que o script possa ser executado do zero quantas vezes forem necessárias, sem falhar por objetos já existentes.

## 12 CONCLUSAO

O projeto KuraVet demonstra, em um cenario aplicado de saude preventiva para pets, a construcao completa de um banco de dados relacional Oracle com integridade referencial, modelagem normalizada ate a 3FN, carga de dados parametrizada e auditavel, consultas analiticas com juncoes agrupadas, leitura sequencial de dados e relatorios com cursor explicito e tomada de decisao. Esta entrega corrige integralmente os itens apontados no feedback da Sprint 2, mantendo tambem os objetos ja validados anteriormente (Funcao JSON manual, Procedimento de subtotais, Funcao de idade do pet e Trigger de auditoria).

O script completo (2TDSPO\_2024\_CodigoSql\_Integrantes.sql) esta organizado em blocos numerados e comentados, cobrindo desde a limpeza de objetos ate os blocos de demonstracao de cada requisito, com tratamento de execucao consistente em toda a base de codigo.

### 12.1 Aderencia ao checklist de avaliacao da Sprint 3

Quadro 9 — Checklist de aderencia aos criterios de avaliacao do feedback do professor

| Item avaliado (pontuacao)                                           | Status                                                                |
|---------------------------------------------------------------------|-----------------------------------------------------------------------|
| Modelagem — DER Barker e MER 3FN (10 pts)                           | Atendido — Secao 3; diagramas oficiais gerados no Oracle Data Modeler |
| Scripts DDL executados no Oracle (10 pts)                           | Atendido — Secao 4; 7 tabelas e 5 sequences criadas com sucesso       |
| Carga de dados via Procedures parametrizadas (20 pts)               | Atendido — Secao 5; 5 procedures, sem hard-code, com log de erros     |
| 2 blocos com $\geq 3$ consultas JOIN + GROUP BY + ORDER BY (20 pts) | Atendido — Secao 6                                                    |
| Bloco valor atual/anterior/proximo, min. 5 linhas (20 pts)          | Atendido — Secao 7; 10 linhas exibidas                                |
| 4 relatorios com cursor explicito e decisao (20 pts)                | Atendido — Secao 8; inclui relatorio completo sumarizado              |
| EXCEPTION WHEN OTHERS + 2 tratamentos por bloco                     | Atendido em todos os blocos — Secao 11                                |
| Folha de rosto com nomes e RMs em ordem alfabetica                  | Atendido — folha de rosto ABNT                                        |
| Organizacao do arquivo .sql                                         | Atendido — script unico, numerado e comentado por secao               |

Fonte: elaborado pelos autores (2026).

**Pendencia para fechamento total da nota maxima: anexar, nas Secoes 6, 7 e 8, os prints de execucao reais dos blocos de consultas com JOIN, do bloco valor atual/anterior/proximo e dos 4 relatorios com cursor explicito, capturados apos rodar o**

**arquivo 2TDSPO\_2024\_CodigoSql\_Integrantes.sql no ambiente Oracle do grupo. Todas as demais seções (Modelagem, Scripts DDL, Carga de Dados, Funções/Procedimentos e Trigger de Auditoria) já possuem evidência real de execução anexada.**